

Cryptographic Primitives of the Swiss Post Voting System

Pseudocode Specification

Swiss Post

Version 1.4.2

Abstract

Cryptographic algorithms play a pivotal role in the Swiss Post Voting System: ensuring their faithful implementation is crucially important. This document provides a mathematically precise and unambiguous specification of some cryptographic primitives underpinning the Swiss Post Voting System. It focuses on the elements common to the system and its verifier, such as the verifiable mix net and non-interactive zero-knowledge proofs. We provide technical details about encoding methods between basic data types and describe each algorithm in pseudocode format.

Disclaimer

E-Voting Community Program Material - please follow our [Code of Conduct](#) describing what you can expect from us, the Coordinated Vulnerability Disclosure Policy, and the contributing guidelines.

Revision chart

Version	Description	Author	Reviewer	Date
0.9	First version for external review	TH, OE	CK, HR, BS, KN	2021-02-05
0.9.1	Minor corrections in existing algorithms	TH, OE	CK, HR, BS, KN	2021-02-12
0.9.2	Completed mix net specification	TH, OE	CK, HR, BS, KN	2021-02-19
0.9.3	Version with reviewers' feedback for publication	TH, OE	CK, HR, BS, KN	2021-03-18
0.9.4	See change log crypto-primitives release 0.8	TH, OE	CK, HR	2021-04-22
0.9.5	See change log crypto-primitives release 0.9	TH, OE	CK, HR, BS	2021-06-22
0.9.6	See change log crypto-primitives release 0.10	TH, OE	CK, HR	2021-07-26
0.9.7	See change log crypto-primitives release 0.11	TH, OE	CK, HR	2021-09-01
0.9.8	See change log crypto-primitives release 0.12	TH, OE	CK, HR, BS	2021-10-15
0.9.9	See change log crypto-primitives release 0.13	TH, HR, OE	CK, BS	2022-02-17
0.9.10	See change log crypto-primitives release 0.14	TH, HR, OE	CK, BS	2022-04-18
1.0.0	See change log crypto-primitives release 0.15	TH, HR, OE	CK, BS	2022-06-24
1.0.1	See change log crypto-primitives release 1.0	TH, HR, OE	CK, BS	2022-10-03
1.1.0	See change log crypto-primitives release 1.1	TH, HR, OE	CK, BS	2022-10-31
1.2.0	See change log crypto-primitives release 1.2	AH, OE	CK, TH	2022-12-09
1.2.1	See change log crypto-primitives release 1.2.1	AH, OE	CK, TH	2023-02-23
1.3.0	See change log crypto-primitives release 1.3.0	AH, OE	CK, TH	2023-04-13
1.3.1	See change log crypto-primitives release 1.3.1	AH, OE	CK, TH	2023-06-15
1.4.0	See change log crypto-primitives release 1.4.0	AH, OE	CK	2024-02-14
1.4.1	See change log crypto-primitives release 1.4.1	AH, OE	CK	2024-06-17
1.4.2	See change log crypto-primitives release 1.4.2	AH, OE	CK	2025-05-16

Contents

Symbols	5
1 Introduction	6
1.1 The Specification of Cryptographic Primitives	6
1.2 Validating the Cryptographic Algorithm's Correctness	7
2 Security Level	8
3 Basic Data Types	10
3.1 Byte Arrays	10
3.2 Integers	14
3.3 Strings	16
4 Alphabets	20
4.1 User-Friendly Alphabet for Codes	20
4.2 Extended Latin Alphabet	20
5 Basic Algorithms	22
5.1 Randomness	22
5.2 Recursive Hash	25
5.3 Hash and Square	29
5.4 KDF	30
5.5 Argon2	32
6 Symmetric Authenticated Encryption	34
7 Digital Signatures	37
7.1 Generating a Signing Key and Certificate	38
7.2 Importing a Trusted Certificate	40
7.3 Signing a Message	41
7.4 Verifying a Message	42
8 ElGamal Cryptosystem	43
8.1 Primality Testing	43
8.2 Parameters Generation	43
8.3 Prime Selection	46
8.4 Key Pair Generation	48
8.5 Encryption	49
8.6 Ciphertext Operations	50
8.7 Decryption	52
8.8 Combining ElGamal Multi-Recipient Public Keys	55

9	Mix Net	56
9.1	Pre-Requisites	59
9.1.1	Shuffle	59
9.1.2	Matrix Dimensions	61
9.2	Commitments	62
9.3	Arguments	66
9.3.1	Shuffle Argument	68
9.3.2	Multi-Exponentiation Argument	72
9.3.3	Product Argument	76
9.3.4	Hadamard Argument	79
9.3.5	Zero Argument	82
9.3.6	Single Value Product Argument	85
10	Zero-Knowledge Proofs	87
10.1	Introduction	87
10.2	Schnorr Proof	88
10.3	Decryption Proof	91
10.4	Exponentiation Proof	94
10.5	Plaintext Equality Proof	97
	References	103
	List of Algorithms	104
	List of Figures	106
	List of Tables	106

Symbols

$\mathbb{A}_{10}$	Alphabet of decimal numbers
$\mathbb{A}_{Base16}$	Base16 (Hex) alphabet [19]
$\mathbb{A}_{Base32}$	Base32 alphabet, including the padding character = [19]
$\mathbb{A}_{Base64}$	Base64 alphabet, including the padding character = [19]
$\mathbb{A}_{UCS}$	Alphabet of the Universal Coded Character Set (UCS) according to ISO/IEC10646
$\mathcal{B}$	Set of possible values for a byte
$\mathcal{B}^*$	Set of byte arrays of arbitrary length
$\mathbb{B}^n$	Set of bit arrays of length n
$\mathbb{N}$	Set of non-negative integer numbers including 0
$\mathbb{N}^+$	Set of strictly positive integer numbers
$\mathbb{P}$	Set of prime numbers
$\mathbb{Z}_p$	Set of integers modulo p
$\mathbb{Z}_q$	Set of integers modulo q
$\mathbb{G}_q$	Set of quadratic residues modulo p , which forms a group of order q
$\mathbb{H}_\ell$	Ciphertext domain $(= \underbrace{\mathbb{G}_q \times \dots \times \mathbb{G}_q}_{\ell+1 \text{ times}})$
$\mathbb{C}_\nu$	Commitment key domain $(= (\mathbb{G}_q \setminus \{1, g\})^{\nu+1})$
p	Encryption group modulus, a large safe prime (exact bitlength defined in the relevant section)
q	Encryption group cardinality s.t. $p = 2 \cdot q + 1$. A large prime (exact bit length defined in the relevant section)
g	Generator of the encryption group
$ x $	Bit length of the number x
$\top$	Truth value true or successful termination
$\perp$	Truth value false or unsuccessful termination

1 Introduction

Switzerland has a longstanding tradition of direct democracy, allowing Swiss citizens to vote approximately four times a year on elections and referendums. In recent years, voter turnout hovered below 40 percent [11].

The vast majority of voters in Switzerland fill out their paper ballots at home and send them back to the municipality by postal mail, usually days or weeks ahead of the actual election date. Remote online voting (referred to as e-voting in this document) would provide voters with some advantages. First, it would guarantee the timely arrival of return envelopes at the municipality (especially for Swiss citizens living abroad). Second, it would improve accessibility for people with disabilities. Third, it would eliminate the possibility of an invalid ballot when inadvertently filling out the ballot incorrectly.

In the past, multiple cantons offered e-voting to a part of their electorate. Many voters would welcome the option to vote online - provided the e-voting system protects the integrity and privacy of their vote [12].

State-of-the-art e-voting systems alleviate the practical concerns of mail-in voting and, at the same time, provide a high level of security. Above all, they must display three properties [32]:

- Individual verifiability: allow a voter to convince herself that the system correctly registered her vote
- Universal verifiability: allow an auditor to check that the election outcome corresponds to the registered votes
- Vote secrecy: do not reveal a voter's vote to anyone

Following these principles, the Federal Chancellery defined stringent requirements for e-voting systems. The Ordinance on Electronic Voting (VEleS - Verordnung über die elektronische Stimmabgabe) and its technical annex (VEleS annex) [8] describes these requirements.

Swiss democracy deserves an e-voting system with excellent security properties. Swiss Post is thankful to all security researchers for their contributions and the opportunity to improve the system's security guarantees. We look forward to actively engaging with academic experts and the hacker community to maximize public scrutiny of the Swiss Post Voting System.

1.1 The Specification of Cryptographic Primitives

A vital element of a trustworthy and robust e-voting system is the description of the cryptographic algorithms in a form that leaves no room for interpretation and minimizes implementation errors [14].

Our pseudocode description of the cryptographic algorithms—inspired by [15]—follows a consistent pattern:

- We display **algorithms** in font without serif and **vectors** in **boldface** or accented by a right-arrow;
- we prefix deterministic algorithms with **Get*** and probabilistic algorithms with **Gen***;
- we designate values that do not change between runs as **Context** and variable values as **Input**;
- we ensure that each algorithm does only one thing (single responsibility principle);
- we explicit domains and ranges of input and output values. We assume that the implementation ensures the correct domain of the input and context elements. E.g. this means that when an input has the expected form $\mathbf{x} = (x_0, \dots, x_{n-1}) \in (\mathbb{G}_q)^n$, the implementation checks that the input elements have the correct form: That $\mathbf{x}$ has exactly n elements and each one is a group member, for instance by calculating that the Jacobi Symbol of each element of $\mathbf{x}$ equals 1.
- we use the range notation for loops such as $i \in [0, n)$. We include the lower bound but exclude the upper bound, i.e. $0 \leq i < n$;
- we use 0-based indexing to close the representational gap between mathematics and code;
- we use **Require** for preconditions and **Ensure** for post-conditions;
- we use **return** to indicate a potentially early termination of the algorithm with the succeeding variable as the returned value; we use **Output** to describe the values that the algorithm produces;

Furthermore, we believe that a specification encompassing the common elements between the Swiss Post Voting System and its verifier (an open-source software verifying the correct establishment of the election result) benefits both systems.

1.2 Validating the Cryptographic Algorithm's Correctness

We augment our specification with test values obtained from an independent implementation of the pseudocode algorithms: our code validates against these test values to increase our confidence in the implementation's correctness. The specification embeds the test values as JSON files within the document.

2 Security Level

Table 2 describes a *testing-only* and a *standard* security level and the associated security parameter selection. The *standard* security level is in line with common cryptographic standards [1, Table 4] while ensuring an acceptable performance. By default, we use the *standard* security level. The *testing-only* security level can be used in unit tests to speed up their execution but must not be used in a productive environment.

Security Level Name	<i>testing-only</i>	<i>standard</i>
Security Strength	-	$\lambda = 128$
Group Parameters	$ p = 8n, n \in \mathbb{N}^{+*}$ $ q = p - 1$	$ p = 3072$ bits $ q = 3071$ bits

Table 2: Security levels

Table 3 defines which primitives and parametrization must be used by different algorithms of the crypto primitives specification. The primitives and their parametrization are used irregardless of the security level chosen.

Cryptographic hash function used in RecursiveHash (algorithm 5.5)	SHA3-256
Extendable output function used in RecursiveHashOfLength (algorithm 5.7)	SHAKE-256 [10] with minimum XOF size $\ell^* = 512$
Hash function used in KDF (algorithm 5.9)	SHA-256
Symmetric algorithm	AES-GCM-256 with nonce size 12 bytes
Signature algorithm	RSASSA-PSS [27]
Signature key size	3072 bits
Underlying hash function and hash for the mask generation function	SHA-256
Mask generation function for PSS	MGF1 [27, B.2.]
Length of the salt	32 bytes
Trailer field number	1, representing the trailer field with value 0xbc [27, A.2.3.]
Parametrization of the password-based key derivation function (Argon2)	See section 5.5

Table 3: Primitives and their parametrization, independent of the security level chosen

3 Basic Data Types

We build upon basic data types such as bytes, integers, strings, and arrays. Moreover, we require algorithms to concatenate and truncate strings and byte arrays, to test primality and to sort arrays.

3.1 Byte Arrays

We denote a byte array B of length n as $\langle b_0, b_1, \dots, b_{n-1} \rangle$ where $b_i \in [\langle 0x00 \rangle, \langle 0xFF \rangle]$ denotes the $i + 1$ -th byte of the array. Byte arrays can be encoded as strings, and, conversely, decoded from strings using Base16, Base32, and Base64 encodings according to RFC4648 [19]. Table 4 shows different examples of byte arrays.

Byte Array	Byte Array (binary form)	Base64	Base32
$\langle 0xF3, 0x01, 0xA3 \rangle$	11110011 00000001 10100011	"8wGj"	"6MA2G==="
$\langle 0xAC \rangle$	10101100	"rA=="	"VQ=====
$\langle 0x1F, 0x7F, 0x9D, 0x15, 0x12 \rangle$	00011111 01111111 10011101 00010101 00010010	"H3+dFRI="	"D57Z2FIS"

Table 4: Example representations of different byte arrays

We indicate concatenation of byte arrays with the $\parallel$ operator.
 $\langle 0xF3, 0x01, 0xA3 \rangle \parallel \langle 0xAC \rangle = \langle 0xF3, 0x01, 0xA3, 0xAC \rangle$

In some cases, we need to cut a byte array to a given bit length, for instance to limit the number of iterations for algorithms such as algorithm 5.1. This is achieved in algorithm 3.1 by taking the low bytes of the given byte array, and applying a bitmask to the first byte taken so that the necessary leading bits are zeroed.

Algorithm 3.1 CutToBitLength: Cuts the given byte array to the requested bit length

Input:

Byte array $B \in \mathcal{B}^N$ s.t. $N \in \mathbb{N}^+$

Requested length in bits $n \in \mathbb{N}^+$

Require: $n \leq N \cdot 8$

▷ This should only be used to cut leading bits

Operation:

```

1: length  $\leftarrow \lceil \frac{n}{8} \rceil$ 
2: offset  $\leftarrow N - \text{length}$ 
3: if  $n \bmod 8 \neq 0$  then
4:    $B'_0 \leftarrow B_{\text{offset}} \wedge (2^{(n \bmod 8)} - 1)$  ▷ Apply the bitwise-AND operator to mask out
      excess bits in the first byte
5: else
6:    $B'_0 \leftarrow B_{\text{offset}}$ 
7: end if
8: for  $i \in [1, \text{length})$  do
9:    $B'_i \leftarrow B_{\text{offset}+i}$ 
10: end for
11: return  $(B'_0, \dots, B'_{\text{length}-1})$ 

```

Output:

$(B'_0, \dots, B'_{\text{length}-1}) \in \mathcal{B}^*$

Test values for algorithm 3.1 are provided in the attached [cut-to-bit-length.json](#) file.

Algorithms 3.2, 3.3, 3.4, 3.5, 3.6 and 3.7 encode and decode byte arrays to and from Base16, Base32 and Base64 encodings. We refer to “standard” Base32 and Base64 encoding; we do *not* use Base64 with URL and filename safe alphabet and Base32 with extended hex alphabet. Potentially, decoding Base32 and Base64 may fail since the encoding is not bijective (only injective). For instance, one cannot decode the string “==TEOD8=” even though it is within the required alphabet.

Algorithm 3.2 Base16Encode

Input:

Byte array $B \in \mathcal{B}^*$

Operation:

1: $S \leftarrow \text{Base16}(B)$

Output:

String $S \in \mathbb{A}_{\text{Base16}}^*$

▷ According to RFC4648 [19]

Algorithm 3.3 Base16Decode

Input:

String $S \in \mathbb{A}_{\text{Base16}}^*$

▷ According to RFC4648 [19]

Operation:

1: $B \leftarrow \text{Base16}^{-1}(S)$

Output:

Byte array $B \in \mathcal{B}^*$

Algorithm 3.4 Base32Encode

Input:

Byte array $B \in \mathcal{B}^*$

Operation:

1: $S \leftarrow \text{Base32}(B)$

Output:

String $S \in \mathbb{A}_{\text{Base32}}^*$

▷ According to RFC4648 [19]

Algorithm 3.5 Base32Decode

Input:

String $S \in \mathbb{A}_{Base32}^*$

▷ According to RFC4648 [19]

Require: S is valid Base32

Operation:

1: $B \leftarrow \text{Base32}^{-1}(S)$

Output:

Byte array $B \in \mathcal{B}^*$

Algorithm 3.6 Base64Encode

Input:

Byte array $B \in \mathcal{B}^*$

Operation:

1: $S \leftarrow \text{Base64}(B)$

Output:

String $S \in \mathbb{A}_{Base64}^*$

▷ According to RFC4648 [19]

Algorithm 3.7 Base64Decode

Input:

String $S \in \mathbb{A}_{Base64}^*$

Require: S is valid Base64

Operation:

1: $B \leftarrow \text{Base64}^{-1}(S)$

Output:

Byte array $B \in \mathcal{B}^*$

3.2 Integers

When converting integers to byte array, we represent them in big-endian byte order. Since we only work with non-negative integers, we treat them as unsigned integers. Table 5 provides some example integers.

Integer	Byte Array (Hex)
0	<0x00>
3	<0x03>
128	<0x80>
23 591	<0x5C, 0x27>
23 592	<0x5C, 0x28>
4 294 967 295	<0xFF, 0xFF, 0xFF, 0xFF>
4 294 967 296	<0x01, 0x00, 0x00, 0x00, 0x00>

Table 5: Example representations of different integers. We use spaces to separate thousands groups.

Therefore, we ignore leading zeros (with an exception for the value 0) and define algorithm 3.8 to convert byte arrays to integers and algorithm 3.9 to convert integers to byte arrays. We avoid the empty byte array $\langle \rangle$ and represent 0 as $\langle 0x00 \rangle$.

$|x|$ derives the minimal bit length of a non-negative integer, e.g. $|4\,294\,967\,295| = 32$ and $|4\,294\,967\,296| = 33$.

Algorithm 3.8 ByteArrayToInteger

Input:

Byte array $B = \langle b_0, b_1, \dots, b_{n-1} \rangle \in \mathcal{B}^n$ of length $n \in \mathbb{N}^+$

Operation:

- 1: $x \leftarrow 0$
 - 2: **for** $i \in [0, n)$ **do**
 - 3: $x \leftarrow 256 \cdot x + b_i$
 - 4: **end for**
-

Output:

$x \in \mathbb{N}$

Algorithm 3.9 IntegerToByteArray

Input:

Integer $x \in \mathbb{N}$

Operation:

- 1: $n \leftarrow \text{ByteLength}(x)$ $\triangleright$ Derive minimal length n of byte array; See algorithm 3.11
 - 2: **for** $i \in [0, n)$ **do**
 - 3: $b_{n-i-1} \leftarrow x \bmod 256$
 - 4: $x \leftarrow \lfloor \frac{x}{256} \rfloor$
 - 5: **end for**
 - 6: $B \leftarrow \langle b_0, b_1, \dots, b_{n-1} \rangle$
-

Output:

Byte array $B \in \mathcal{B}^n$

Algorithm 3.10 allows to set the desired length of the byte array when converting an integer to a byte array.

Algorithm 3.10 IntegerToFixedLengthByteArray

Input:

Integer $x \in \mathbb{N}$

Requested length in bytes $n \in \mathbb{N}$

Require: $\text{ByteLength}(x) \leq n$

Operation:

- 1: $m \leftarrow \text{ByteLength}(x)$ $\triangleright$ Derive minimal length m of byte array; See algorithm 3.11
 - 2: **for** $i \in [0, n - m)$ **do**
 - 3: $b_i \leftarrow 0$
 - 4: **end for**
 - 5: **for** $i \in [0, m)$ **do**
 - 6: $b_{n-i-1} \leftarrow x \bmod 256$
 - 7: $x \leftarrow \lfloor \frac{x}{256} \rfloor$
 - 8: **end for**
 - 9: $B \leftarrow \langle b_0, b_1, \dots, b_{n-1} \rangle$
-

Output:

Byte array $B \in \mathcal{B}^n$

We define algorithm 3.11 to compute the length of the byte representation of an integer.

Algorithm 3.11 ByteLength: Compute the length of the byte representation of an integer

Input:

Integer $x \in \mathbb{N}$

Operation:

1: $n \leftarrow \left\lceil \frac{|x|}{8} \right\rceil$

2: $n \leftarrow \max(n, 1)$ $\triangleright$ Assuming the byte representation of 0 is $\langle 0x00 \rangle$ with length 1

Output:

Byte length $n \in \mathbb{N}^+$

3.3 Strings

We encode strings in the universal coded character set (UCS) as defined in ISO/IEC10646, which is used by the encoding format UTF-8 (see RFC3629 [35]). Table 6 highlights some examples.

String	Byte Array (UCS)
“ABC”	$\langle 0x41, 0x42, 0x43 \rangle$
“Ä”	$\langle 0xC3, 0x84 \rangle$
“1001”	$\langle 0x31, 0x30, 0x30, 0x31 \rangle$
“1A”	$\langle 0x31, 0x41 \rangle$

Table 6: Example representations of different strings

Algorithms 3.12 and 3.13 convert byte arrays to strings and vice versa. Potentially, the `ByteArrayToString` method can fail since not every byte array is a valid UTF-8 encoding.

Algorithm 3.12 `StringToByteArray`

Input:

String $S \in \mathbb{A}_{UCS}^*$

Operation:

1: $B \leftarrow \text{UTF-8}(S)$ ▷ Encode S in UTF-8

Output:

Byte array $B \in \mathcal{B}^*$

Algorithm 3.13 `ByteArrayToString`

Input:

Byte array $B = \langle b_0, b_1, \dots, b_{n-1} \rangle \in \mathcal{B}^n$ of length $n \in \mathbb{N}^+$

Operation:

1: **if** B does not correspond to a valid UTF-8 encoding **then**
2: **return** $\perp$
3: **end if**
4: $S \leftarrow \text{UTF-8}^{-1}(B)$

Output:

String $S \in \mathbb{A}_{UCS}^*$

Moreover, we specify a method `StringToInteger` that translates a decimal `String` representation to an integer. Conversely, the algorithm `IntegerToString` converts integers to `Strings`. Beware that the method `StringToInteger(String)` yields a different result than the conversion `ByteArrayToInteger(StringToByteArray(String))`.

Table 7 highlights some examples.

String	Integer
"0"	0
"1"	1
"1001"	1001
"0021"	21
"1A"	$\perp$

Table 7: Example conversions of strings to integers

Algorithm 3.14 `StringToInteger`

Input:

String $S \in \mathbb{A}_{10}^n, n \in \mathbb{N}^+$

Operation:

- 1: **if** S is not valid decimal representation **then**
 - 2: **return** $\perp$
 - 3: **end if**
 - 4: $x \leftarrow \text{Decimal}(S)$ $\triangleright$ Convert the String into its decimal representation (radix = 10)
-

Output:

Non-negative integer $x \in \mathbb{N}$

Algorithm 3.15 `IntegerToString`

Input:

Non-negative integer $x \in \mathbb{N}$

Operation:

- 1: $S \leftarrow \text{Decimal}^{-1}(x)$ $\triangleright$ Convert the integers' decimal representation (radix = 10) into a String
-

Output:

String $S \in \mathbb{A}_{10}^n, n \in \mathbb{N}^+$

The method **Truncate** truncates a string to the desired maximum length if the string is larger than the desired maximum length.

Algorithm 3.16 Truncate

Input:

String $S \in \mathbb{A}^u, u \in \mathbb{N}^+$

Desired maximum length of string: $\ell \in \mathbb{N}^+$

Operation:

- 1: $m \leftarrow \min(u, \ell)$
 - 2: **for** $i \in [0, m)$ **do**
 - 3: $S'_i \leftarrow S_i$
 - 4: **end for**
 - 5: $S' = \langle S'_0, \dots, S'_{m-1} \rangle$
-

Output:

The truncated string $S' \in \mathbb{A}^m$

4 Alphabets

We define two alphabets which are useful for e-voting.

4.1 User-Friendly Alphabet for Codes

For codes that have to be entered by humans, it is important to choose a user-friendly alphabet. The alphabet used should include only letters and numbers easily found on any keyboard, and the chance of misspelling should be minimized by excluding letters and/or numbers that look similar. We define an alphabet $\mathbb{A}_{u32}$, which is an adjusted version of the Base32 alphabet, optimized for usability. Compared to the Base32 alphabet, we omit the letters "l" and "o" and use instead "8" and "9". Further, the padding character "=" of the Base32 alphabet is omitted and lowercase letters are used instead of uppercase letters. This results in the following alphabet: $\mathbb{A}_{u32} = (a, b, c, d, e, f, g, h, i, j, k, m, n, p, q, r, s, t, u, v, w, x, y, z, 2, 3, 4, 5, 6, 7, 8, 9)$, of size $|\mathbb{A}_{u32}| = 32$.

4.2 Extended Latin Alphabet

We define an extended Latin alphabet including letters used in different Latin languages as well as some symbols and numbers. The following list defines the alphabet which we will refer to as $\mathbb{A}_{\text{latin}}$. Each symbol is given with their UTF-8 codepoint to remove ambiguity. The size of the alphabet defined is $|\mathbb{A}_{\text{latin}}| = 141$. In an e-voting context, the alphabet can be useful if the election allows write-ins.

- | | | |
|----------------------------|----------------------------|----------------------------|
| • # (U+0023) $\mapsto$ 000 | • 7 (U+0037) $\mapsto$ 016 | • N (U+004E) $\mapsto$ 032 |
| • (U+0020) $\mapsto$ 001 | • 8 (U+0038) $\mapsto$ 017 | • O (U+004F) $\mapsto$ 033 |
| • ' (U+0027) $\mapsto$ 002 | • 9 (U+0039) $\mapsto$ 018 | • P (U+0050) $\mapsto$ 034 |
| • ((U+0028) $\mapsto$ 003 | • A (U+0041) $\mapsto$ 019 | • Q (U+0051) $\mapsto$ 035 |
| •) (U+0029) $\mapsto$ 004 | • B (U+0042) $\mapsto$ 020 | • R (U+0052) $\mapsto$ 036 |
| • , (U+002C) $\mapsto$ 005 | • C (U+0043) $\mapsto$ 021 | • S (U+0053) $\mapsto$ 037 |
| • - (U+002D) $\mapsto$ 006 | • D (U+0044) $\mapsto$ 022 | • T (U+0054) $\mapsto$ 038 |
| • . (U+002E) $\mapsto$ 007 | • E (U+0045) $\mapsto$ 023 | • U (U+0055) $\mapsto$ 039 |
| • / (U+002F) $\mapsto$ 008 | • F (U+0046) $\mapsto$ 024 | • V (U+0056) $\mapsto$ 040 |
| • 0 (U+0030) $\mapsto$ 009 | • G (U+0047) $\mapsto$ 025 | • W (U+0057) $\mapsto$ 041 |
| • 1 (U+0031) $\mapsto$ 010 | • H (U+0048) $\mapsto$ 026 | • X (U+0058) $\mapsto$ 042 |
| • 2 (U+0032) $\mapsto$ 011 | • I (U+0049) $\mapsto$ 027 | • Y (U+0059) $\mapsto$ 043 |
| • 3 (U+0033) $\mapsto$ 012 | • J (U+004A) $\mapsto$ 028 | • Z (U+005A) $\mapsto$ 044 |
| • 4 (U+0034) $\mapsto$ 013 | • K (U+004B) $\mapsto$ 029 | • a (U+0061) $\mapsto$ 045 |
| • 5 (U+0035) $\mapsto$ 014 | • L (U+004C) $\mapsto$ 030 | • b (U+0062) $\mapsto$ 046 |
| • 6 (U+0036) $\mapsto$ 015 | • M (U+004D) $\mapsto$ 031 | • c (U+0063) $\mapsto$ 047 |

• d (U+0064) $\mapsto$ 048	• Å (U+00C0) $\mapsto$ 079	• à (U+00E0) $\mapsto$ 110
• e (U+0065) $\mapsto$ 049	• Á (U+00C1) $\mapsto$ 080	• á (U+00E1) $\mapsto$ 111
• f (U+0066) $\mapsto$ 050	• Â (U+00C2) $\mapsto$ 081	• â (U+00E2) $\mapsto$ 112
• g (U+0067) $\mapsto$ 051	• Ã (U+00C3) $\mapsto$ 082	• ã (U+00E3) $\mapsto$ 113
• h (U+0068) $\mapsto$ 052	• Ä (U+00C4) $\mapsto$ 083	• ä (U+00E4) $\mapsto$ 114
• i (U+0069) $\mapsto$ 053	• Å (U+00C5) $\mapsto$ 084	• å (U+00E5) $\mapsto$ 115
• j (U+006A) $\mapsto$ 054	• Æ (U+00C6) $\mapsto$ 085	• æ (U+00E6) $\mapsto$ 116
• k (U+006B) $\mapsto$ 055	• Ç (U+00C7) $\mapsto$ 086	• ç (U+00E7) $\mapsto$ 117
• l (U+006C) $\mapsto$ 056	• È (U+00C8) $\mapsto$ 087	• è (U+00E8) $\mapsto$ 118
• m (U+006D) $\mapsto$ 057	• É (U+00C9) $\mapsto$ 088	• é (U+00E9) $\mapsto$ 119
• n (U+006E) $\mapsto$ 058	• Ê (U+00CA) $\mapsto$ 089	• ê (U+00EA) $\mapsto$ 120
• o (U+006F) $\mapsto$ 059	• Ë (U+00CB) $\mapsto$ 090	• ë (U+00EB) $\mapsto$ 121
• p (U+0070) $\mapsto$ 060	• Ì (U+00CC) $\mapsto$ 091	• ì (U+00EC) $\mapsto$ 122
• q (U+0071) $\mapsto$ 061	• Í (U+00CD) $\mapsto$ 092	• í (U+00ED) $\mapsto$ 123
• r (U+0072) $\mapsto$ 062	• Î (U+00CE) $\mapsto$ 093	• î (U+00EE) $\mapsto$ 124
• s (U+0073) $\mapsto$ 063	• Ï (U+00CF) $\mapsto$ 094	• ï (U+00EF) $\mapsto$ 125
• t (U+0074) $\mapsto$ 064	• Ð (U+00D0) $\mapsto$ 095	• ð (U+00F0) $\mapsto$ 126
• u (U+0075) $\mapsto$ 065	• Ñ (U+00D1) $\mapsto$ 096	• ñ (U+00F1) $\mapsto$ 127
• v (U+0076) $\mapsto$ 066	• Ò (U+00D2) $\mapsto$ 097	• ò (U+00F2) $\mapsto$ 128
• w (U+0077) $\mapsto$ 067	• Ó (U+00D3) $\mapsto$ 098	• ó (U+00F3) $\mapsto$ 129
• x (U+0078) $\mapsto$ 068	• Ô (U+00D4) $\mapsto$ 099	• ô (U+00F4) $\mapsto$ 130
• y (U+0079) $\mapsto$ 069	• Õ (U+00D5) $\mapsto$ 100	• õ (U+00F5) $\mapsto$ 131
• z (U+007A) $\mapsto$ 070	• Ö (U+00D6) $\mapsto$ 101	• ö (U+00F6) $\mapsto$ 132
• ¢ (U+00A2) $\mapsto$ 071	• Ø (U+00D8) $\mapsto$ 102	• ø (U+00F8) $\mapsto$ 133
• Š (U+0160) $\mapsto$ 072	• Ù (U+00D9) $\mapsto$ 103	• ù (U+00F9) $\mapsto$ 134
• š (U+0161) $\mapsto$ 073	• Ú (U+00DA) $\mapsto$ 104	• ú (U+00FA) $\mapsto$ 135
• Ž (U+017D) $\mapsto$ 074	• Û (U+00DB) $\mapsto$ 105	• û (U+00FB) $\mapsto$ 136
• ž (U+017E) $\mapsto$ 075	• Ü (U+00DC) $\mapsto$ 106	• ü (U+00FC) $\mapsto$ 137
• Œ (U+0152) $\mapsto$ 076	• Ý (U+00DD) $\mapsto$ 107	• ý (U+00FD) $\mapsto$ 138
• œ (U+0153) $\mapsto$ 077	• Þ (U+00DE) $\mapsto$ 108	• þ (U+00FE) $\mapsto$ 139
• Ÿ (U+0178) $\mapsto$ 078	• ß (U+00DF) $\mapsto$ 109	• ÿ (U+00FF) $\mapsto$ 140

5 Basic Algorithms

5.1 Randomness

Several algorithms draw a value at random from a given domain and rely on a primitive providing the requested number of independent random bytes. Standard implementations for generating cryptographically secure random bytes¹ are available in most programming languages; therefore, we omit the pseudocode for this primitive and call it `RandomBytes(length)`, where `length` $\in \mathbb{N}$ is the required number of bytes, and the output is in $\mathcal{B}^{\text{length}}$.

Algorithm 5.1 `GenRandomInteger`: Provide a random integer between 0 (incl.) and m (excl.)

Input:

Upper bound $m \in \mathbb{N}^+$

Operation:

```

1: if  $m = 1$  then
    return  $r = 0$ 
2: end if
3:  $\text{length} \leftarrow \text{ByteLength}(m - 1)$  ▷ See algorithm 3.11
4:  $\text{bitLength} \leftarrow |m - 1|$ 
5: do
6:    $\text{rBytes} \leftarrow \text{CutToBitLength}(\text{RandomBytes}(\text{length}), \text{bitLength})$  ▷ See algorithm 3.1
7:    $r \leftarrow \text{ByteArrayToInteger}(\text{rBytes})$  ▷ See algorithm 3.8
8: while  $r \geq m$ 

```

Output:

Random integer $r \in [0, m)$

¹A cryptographically secure random bytes generator has the following characteristics: it is designed for cryptographic use, generates independent, unbiased (i.e. uniform) bytes and relies on a high-quality entropy source [20]

Algorithm 5.2 GenRandomVector: Generate a random vector from $\mathbb{Z}_q^n$

Input:

Exclusive upper bound $q \in \mathbb{N}^+$
Length $n \in \mathbb{N}^+$

Operation:

```

1: for  $i \in [0, n)$  do
2:    $r_i \leftarrow \text{GenRandomInteger}(q)$  ▷ See algorithm 5.1
3: end for

```

Output:

Random vector $(r_0, \dots, r_{n-1}) \in \mathbb{Z}_q^n$

Algorithm 5.3 GenRandomString

Input:

Desired length of string: $l \in \mathbb{N}^+$
Alphabet from which to choose the string: $\mathbb{A} = (S_0, \dots, S_{k-1})$ ▷ k is the number of elements of the alphabet

Operation:

```

1: for  $i \in [0, l)$  do
2:    $m \leftarrow \text{GenRandomInteger}(k)$  ▷ See algorithm 5.1
3:    $S'_i \leftarrow S_m$ 
4: end for
5:  $S' = \langle S'_0, \dots, S'_{l-1} \rangle$ 

```

Output:

String $S' \in \mathbb{A}^l$ ▷ A random string of the given alphabet

Algorithm 5.4 GenUniqueDecimalStrings

Input:

Desired length of each code: $l \in \mathbb{N}^+$

Number of unique codes: $n \in \mathbb{N}^+$

Require: $n \leq 10^l$

Operation:

1: **codes** $\leftarrow ()$

2: **while** $|\text{codes}| < n$ **do**

3: $c \leftarrow \text{GenRandomString}(l, \mathbb{A}_{10})$

▷ See algorithm 5.3

4: **if** $c \notin \text{codes}$ **then**

5: **codes** $\leftarrow \text{codes} \cup \{c\}$

6: **end if**

7: **end while**

Output:

codes $\in (\mathbb{A}_{10}^l)^n$

5.2 Recursive Hash

Our recursive hash function—inspired by CHVote [15]—ensures that different inputs to the hash function result in different outputs. In particular, the recursive hash function provides domain-separation: hashing (“A”, “B”) does not yield the same result as hashing (“AB”).

To prevent collisions across the different possible input domains, we prepend a single byte to the scalar input values, according to their type. This implies that `RecursiveHash` will give a different result for the input string “A” than for byte array `<0x41>`.

The recursive definition of the domain implies that infinite inputs are possible in theory (such as self-referencing inputs), in which case the algorithm does not terminate. In practice, inputs are bound to structures that can be represented in memory.

Algorithm 5.5 RecursiveHash: Computes the hash value of multiple inputs

Context:

Cryptographic hash function $\text{Hash} : \mathcal{B}^* \rightarrow \mathcal{B}^L$, $L \in \mathbb{N}^+$ $\triangleright$ Defined in table 3 $\triangleright$
Outputs a byte array of length L

Input:

Values $v_0, \dots, v_{k-1}$. Each value v_i is in domain $\mathcal{V}$, recursively defined as the union of:

- the set of byte arrays $\mathcal{B}^*$
- the set of valid UCS strings $\mathbb{A}_{UCS}$
- the set of non-negative integers $\mathbb{N}$
- the set of vectors $\mathcal{V}^*$

Require: $k > 0$, $L > 0$

Operation:

```

1: if  $k > 1$  then                                 $\triangleright$  Avoid computing  $\text{Hash}(\text{Hash}(v_0))$  when  $k = 1$ 
2:    $\mathbf{v} \leftarrow (v_0, \dots, v_{k-1})$ 
3:    $d \leftarrow \text{RecursiveHash}(\mathbf{v})$ 
4: else
5:    $w \leftarrow v_0$ 
6:   if  $w \in \mathcal{B}^*$  then
7:      $d \leftarrow \text{Hash}(\text{<0x00>} || w)$ 
8:   else if  $w \in \mathbb{N}$  then
9:      $d \leftarrow \text{Hash}(\text{<0x01>} || \text{IntegerToByteArray}(w))$            $\triangleright$  See algorithm 3.9
10:  else if  $w \in \mathbb{A}_{UCS}$  then
11:     $d \leftarrow \text{Hash}(\text{<0x02>} || \text{StringToByteArray}(w))$            $\triangleright$  See algorithm 3.12
12:  else if  $w = (w_0, \dots, w_j)$  then           $\triangleright$   $w$  might be an empty list  $w = ()$ 
13:     $d \leftarrow \text{Hash}(\text{<0x03>} || \text{RecursiveHash}(w_0) || \dots || \text{RecursiveHash}(w_j))$ 
14:  else
15:    return  $\perp$ 
16: end if

```

Output:

The digest $d \in \mathcal{B}^L$

Test values for algorithm 5.5 are provided in the attached [recursive-hash-sha3-256.json](#) file.

All test files provided in the current document for the algorithms relying on this algorithm assume that the hash function defined in table 3 is used.

In some cases, the output of a hash needs to be uniformly distributed across $\mathbb{Z}_q$. To achieve this, we use an extendable output function and reduce the resulting value modulo q . We draw a value from a domain that is much larger than the target domain in order to have a negligible modulo bias. Algorithm 5.6 deterministically draws new values, relying on algorithm 5.7 to perform the actual hashing. For algorithm 5.7, without loss of generality, we assume a signature for an extendable output function that takes the requested byte length as first parameter, and the input byte array as second parameter.

Algorithm 5.6 RecursiveHashToZq: Computes the hash value of multiple inputs uniformly into $\mathbb{Z}_q$

Context:

The security level defining the security strength λ , according to table 2.

Input:

Exclusive upper bound $q \in \mathbb{N}^+$

Values $v_0, \dots, v_{k-1}$. Each value v_i is in domain $\mathcal{V}$, recursively defined as the union of:

- the set of byte arrays $\mathcal{B}^*$
- the set of valid UCS strings $\mathbb{A}_{UCS}$
- the set of non-negative integers $\mathbb{N}$
- the set of vectors $\mathcal{V}^*$

Require: $k > 0$, $|q| \geq 512$

Operation:

- 1: $h' \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHashOfLength}(|q|+2\lambda, q, \text{"RecursiveHash"}, \mathbf{v})) \triangleright$
See algorithms 3.8 and 5.7
 - 2: $h \leftarrow h' \bmod q$
-

Output:

$h \in \mathbb{Z}_q$

Test values for algorithm 5.6 are provided in the attached [recursive-hash-to-zq.json](#) file.

Algorithm 5.7 RecursiveHashOfLength: Computes the hash value of multiple inputs to a given bit length

Context:

Extendable output function $\text{XOF} : \mathbb{N}^+ \times \mathcal{B}^* \rightarrow \mathcal{B}^u, u \in \mathbb{N}^+$ $\triangleright$ Defined in table 3

Input:

Requested bit length $\ell \in \mathbb{N}^+$

Values $v_0, \dots, v_{k-1}$. Each value v_i is in domain $\mathcal{V}$, recursively defined as the union of:

- the set of byte arrays $\mathcal{B}^*$
- the set of valid UCS strings $\mathbb{A}_{UCS}$
- the set of non-negative integers $\mathbb{N}$
- the set of vectors $\mathcal{V}^*$

Require: $k > 0, \ell \geq \ell^*$ $\triangleright \ell^*$ is the minimum XOF size as defined in table 3

Operation:

```

1:  $L \leftarrow \lceil \ell/8 \rceil$ 
2: if  $k > 1$  then  $\triangleright$  Avoid computing  $\text{Hash}(\text{Hash}(v_0))$  when  $k = 1$ 
3:    $\mathbf{v} \leftarrow (v_0, \dots, v_{k-1})$ 
4:    $d \leftarrow \text{RecursiveHashOfLength}(\ell, \mathbf{v})$ 
5: else
6:    $w \leftarrow v_0$ 
7:   if  $w \in \mathcal{B}^*$  then
8:      $d \leftarrow \text{CutToBitLength}(\text{XOF}(L, \langle 0x00 \rangle || w), \ell)$   $\triangleright$  See algorithm 3.1
9:   else if  $w \in \mathbb{N}$  then
10:     $d \leftarrow \text{CutToBitLength}(\text{XOF}(L, \langle 0x01 \rangle || \text{IntegerToByteArray}(w)), \ell)$   $\triangleright$  See algorithm 3.9
11:   else if  $w \in \mathbb{A}_{UCS}$  then
12:     $d \leftarrow \text{CutToBitLength}(\text{XOF}(L, \langle 0x02 \rangle || \text{StringToByteArray}(w)), \ell)$   $\triangleright$  See algorithm 3.12
13:   else if  $w = (w_0, \dots, w_j)$  then  $\triangleright w$  might be an empty list  $w = ()$ 
14:     for  $i \in [0, j]$  do
15:        $h_i \leftarrow \text{RecursiveHashOfLength}(\ell, w_i)$ 
16:     end for
17:      $d \leftarrow \text{CutToBitLength}(\text{XOF}(L, \langle 0x03 \rangle || h_0 || \dots || h_j), \ell)$ 
18:   else
19:     return  $\perp$ 
20: end if

```

Output:

The digest $d \in \mathcal{B}^L$ $\triangleright$ Where the $8L - \ell$ first bits are set to 0

5.3 Hash and Square

At various places in the protocol, we break the homomorphic properties of certain operations by hashing and squaring values.² Given our choice of group parameters, modular squaring the hash's output ensures that the resulting value is a mathematical group member.

Algorithm 5.8 HashAndSquare: Hashes a value and squares the result

Context:

Group modulus $p \in \mathbb{P}$

Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$

Input:

$x \in \mathbb{N}$

Operation:

- 1: $x_h \leftarrow \text{RecursiveHashToZq}(q, (\text{"HashAndSquare"}, x)) + 1$ ▷ See algorithm 5.6,
avoiding the case $x_h = 0$
 - 2: $y \leftarrow x_h^2 \bmod p$
-

Output:

$y \in \mathbb{G}_q$

²The exponentiation function is a pseudo-random function if the input is randomly distributed.

5.4 KDF

The algorithms below rely on RFC5869 [22] to describe a HMAC-based key derivation function (HKDF) to produce key material from a high-entropy source. The pseudocode algorithms below only use the HKDF-expand part of the RFC, because the system only uses this function with cryptographically strong keys.

We note the function specified in section 2.3 of the RFC as HKDF-Expand, using the Hash function defined in the context of the pseudocode and giving it the following inputs, in this order:

- a pseudo-random key, of length at least equal to the block size of the Hash function;
- additional context info, a byte array of arbitrary length;
- and the required length.

Algorithm 5.9 KDF: Key derivation function using HKDF-expand

Context:

Cryptographic hash function $\text{Hash} : \mathcal{B}^* \rightarrow \mathcal{B}^L$, $L \in \mathbb{N}^+$ $\triangleright$ Defined in table 3 $\triangleright$
Outputs a byte array of length L

Input:

The cryptographically strong pseudo-random key $\text{PRK} \in \mathcal{B}^{l_{\text{key}}}$
Additional context information $(\text{info}_0, \dots, \text{info}_{n-1}) \in (\mathbb{A}_{UCS}^*)^n$, s.t. $n \in \mathbb{N}$
The required byte length $\ell \in \mathbb{N}^+$

Require: $l_{\text{key}} \geq L$

Require: $\ell \leq 255 \cdot L$

Require: $\text{length}(\text{StringToByteArray}(\text{info}_i)) \leq 255 \ \forall i$ $\triangleright$ See algorithm 3.12. The length is the number of bytes in the byte array

Operation:

```

1: info  $\leftarrow \langle \rangle$   $\triangleright$  Start with the empty byte array
2: for  $i \in [0, n)$  do
3:    $\text{info}_{i, \text{bytes}} \leftarrow \text{StringToByteArray}(\text{info}_i)$   $\triangleright$  See algorithm 3.12
4:    $\text{info} \leftarrow \text{info} \parallel \text{length}(\text{info}_{i, \text{bytes}}) \parallel \text{info}_{i, \text{bytes}}$   $\triangleright$  Since the length is restricted to 255 bytes, it is encoded as a single unsigned byte
5: end for
 $\triangleright$  I.e. the empty byte array if  $n = 0$ 
6:  $\text{OKM} \leftarrow \text{HKDF-Expand}(\text{PRK}, \text{info}, \ell)$   $\triangleright$  As per RFC5869 [22], section 2.3
7: return OKM

```

Output:

The output keying material $\text{OKM} \in \mathcal{B}^\ell$

Test values for algorithm 5.9 are provided in the attached [kdf.json](#) file.

Algorithm 5.10 is used when we need the resulting key material to be in $\mathbb{Z}_q$. Similar to algorithm 5.6, we draw a value from a domain that is much larger than the target domain to reduce modulo bias.

Algorithm 5.10 KDFToZq: Use the KDF function to generate a value in $\mathbb{Z}_q$

Context:

The security level defining the security strength λ , according to table 2.

Cryptographic hash function $\text{Hash} : \mathcal{B}^* \rightarrow \mathcal{B}^L$, $L \in \mathbb{N}^+$ $\triangleright$ Defined in table 3 $\triangleright$

Outputs a byte array of length L

Input:

The cryptographically strong pseudo-random key $\text{PRK} \in \mathcal{B}^l$

Additional context information $\text{info} \in (\mathbb{A}_{UCS}^*)^n$, s.t. $n \in \mathbb{N}$

The requested exclusive upper bound $q \in \mathbb{N}^+$

Require: $l \geq L$

Require: $\text{ByteLength}(q) \geq L$

$\triangleright$ See algorithm 3.11

Operation:

1: $\ell \leftarrow \text{ByteLength}(q) + \frac{\lambda}{4}$

$\triangleright$ See algorithm 3.11

2: $h \leftarrow \text{KDF}(\text{PRK}, \text{info}, \ell)$

$\triangleright$ See algorithm 5.9

3: $u \leftarrow \text{ByteArrayToInteger}(h) \bmod q$

$\triangleright$ See algorithm 3.8

Output:

The value $u \in \mathbb{Z}_q$

Test values for algorithm 5.10 are provided in the attached [kdf-to-zq.json](#) file.

5.5 Argon2

Argon2 [4] is a memory-hard key derivation function and represents the state of the art for password storage. Since it can be parametrized for parallelism and memory usage as well as iteration count, it is very efficient at increasing the cost of brute-force attacks, more so than PBKDF based on standard hashing functions, which can be computed more efficiently by attackers with specialized hardware.

We use Argon2 on key material that needs to be entered by humans, to keep the system usable while offsetting the limited entropy by making brute-force attacks more expensive. To avoid making any assumptions on the feasibility of side-channels attacks, we use the **Argon2id** variant, designed for this purpose.

We adhere to the RFC9106 specifications [5] and define three Argon2id profiles. The first profile, **STANDARD**, is the recommended option that utilizes a significant amount of memory. The second profile, **LESS_MEMORY**, is designed for environments with limited memory. The third profile, **TEST**, should only be used for executing quick unit tests with minimal memory usage. In table 8, we summarize the profiles.

Profile	Memory (in KiB)	Parallelism	Iterations
STANDARD	2^{21}	4	1
LESS_MEMORY	2^{16}	4	3
TEST	2^{14}	4	1

Table 8: Profiles for Argon2

We use a fixed tag length of 32 bytes, and a salt length of 16 bytes. As for the parallelism, memory usage and iteration count parameters, these should be chosen taking in consideration the entropy of the input (and thus the need for additional cost) and the maximal acceptable delay for each use.

We distinguish the case where the salt needs to be generated (creation of the reference tag – algorithm 5.11) and the case where the salt is provided as input, to ensure the same tag is generated – algorithm 5.12.

In both cases, we assume without loss of generality the existence of a function **argon2id** : $\text{parameters} \times \mathcal{B}^* \rightarrow \mathcal{B}^*$, where the inputs are the parameters provided to Argon2 and the low-entropy key material and the output is the resulting tag.

Algorithm 5.11 GenArgon2id: Compute Argon2 tag and salt

Context:

Memory usage parameter $m \in [14, 24]$ ▷ See table 8
Parallelism parameter $p \in [1, 16]$
Iteration count $i \in [1, 256]$

Input:

Input keying material $k \in \mathcal{B}^*$

Operation:

1: $s \leftarrow \text{RandomBytes}(16)$
2: $t \leftarrow \text{GetArgon2id}(k, s)$ ▷ See algorithm 5.12
3: **return** (t, s)

Output:

The tag and the salt: $(t, s) \in \mathcal{B}^{32} \times \mathcal{B}^{16}$

Test values are provided in [gen-argon2id.json](#).

Algorithm 5.12 GetArgon2id: Compute Argon2 tag

Context:

Memory usage parameter $m \in [14, 24]$ ▷ See table 8
Parallelism parameter $p \in [1, 16]$
Iteration count $i \in [1, 256]$

Input:

Input keying material $k \in \mathcal{B}^*$
The salt $s \in \mathcal{B}^{16}$

Operation:

1: $c \leftarrow \{$
 $\text{tagLength} : 32,$
 $\text{salt} : s,$
 $\text{memory} : 2^m,$ ▷ Given in KiB, thus $m = 21$ implies a memory usage of 2GiB
 $\text{parallelism} : p,$
 $\text{iterations} : i$
 $\}$
2: $t \leftarrow \text{argon2id}(c, k)$

Output:

The tag $t \in \mathcal{B}^{32}$

Test values are provided in [get-argon2id.json](#).

6 Symmetric Authenticated Encryption

We define a symmetric authenticated encryption scheme based on Authenticated Encryption with Associated Data (AEAD), as defined in RFC5116 [24].

We denote the function specified in [24, section 2.1] as `AuthenticatedEncryption` with the following inputs in this order:

- a **secret key**, a byte array of length k ;
- a **nonce**, a byte array of length n ;
- a **plaintext** with the data to be encrypted, a byte array of length p which may be zero;
- the **associated data** with the data to be authenticated but not encrypted, a byte array which may be of length zero;

It produces a single output **ciphertext**, a byte array at least as long as the plaintext.

We denote the function specified in [24, section 2.2] as `AuthenticatedDecryption` with the following inputs (as defined above) in this order: **secret key**, **nonce**, **associated data**, **ciphertext**. It produces a single output, either **plaintext** or $\perp$.

For the implementation we use AES-GCM-256 as the AEAD algorithm (see table 3) with $n = 12$ bytes (as described in [24] and recommended in [18]) and $p \leq 64 \times 10^9$ bytes ([9] section 3). We use a randomized nonce as described in [9] section 8.2.2, with the given limitation that “the total number of invocations of the authenticated encryption function shall not exceed 2^{32} ” for a given key. Moreover, it is critical that nonces should not be reused, otherwise the security properties break down. In particular, algorithm ?? should not be used in a virtualized environment, as a rollback could lead to a nonce reuse. Callers of these algorithms should make sure that the preceding properties are respected.

Algorithm 6.1 GenCiphertextSymmetric: Symmetric authenticated encryption

Context:

Authenticated encryption function $\text{AuthenticatedEncryption} : (\mathcal{B}^k, \mathcal{B}^n, \mathcal{B}^p, \mathcal{B}^*) \rightarrow \mathcal{B}^c$ s.t. $k \in \mathbb{N}^+, n \in \mathbb{N}^+, p \in \mathbb{N}, c \in \mathbb{N}^+$ with the constraints on k, n, p of the specific algorithm used

Input:

The encryption key $K \in \mathcal{B}^k$

The plaintext $P \in \mathcal{B}^p$

Associated data $(\text{associated}_0, \dots, \text{associated}_{d-1}) \in (\mathbb{A}_{UCS}^*)^d$, s.t. $d \in \mathbb{N}$

Require: $\text{length}(\text{StringToByteArray}(\text{associated}_i)) \leq 255 \ \forall i \triangleright$ The length is the number of bytes in the byte array

Operation:

- 1: $\text{nonce} \leftarrow \text{RandomBytes}(n)$
 - 2: $\text{associated} \leftarrow \langle \rangle \quad \triangleright$ Start with the empty byte array
 - 3: **for** $i \in [0, d)$ **do**
 - 4: $\text{associated}_{i,\text{bytes}} \leftarrow \text{StringToByteArray}(\text{associated}_i) \quad \triangleright$ See algorithm 3.12
 - 5: $\text{associated} \leftarrow \text{associated} \parallel \text{length}(\text{associated}_{i,\text{bytes}}) \parallel \text{associated}_{i,\text{bytes}} \quad \triangleright$ Since the length is restricted to 255, it is encoded as a single unsigned byte
 - 6: **end for**
 - 7: $C \leftarrow \text{AuthenticatedEncryption}(K, \text{nonce}, P, \text{associated}) \quad \triangleright$ As per [24], section 2.1
-

Output:

The authenticated ciphertext $C \in \mathcal{B}^c$

The nonce $\text{nonce} \in \mathcal{B}^n$

Test values are provided in the [gen-ciphertext-symmetric.json](#) file.

Algorithm 6.2 GetPlaintextSymmetric: Symmetric authenticated decryption

Context:

Authenticated decryption function $\text{AuthenticatedDecryption} : (\mathcal{B}^k, \mathcal{B}^n, \mathcal{B}^*, \mathcal{B}^c) \rightarrow \mathcal{B}^p$,
 $k \in \mathbb{N}^+, n \in \mathbb{N}^+, p \in \mathbb{N}, c \in \mathbb{N}^+$ with the constraints on k, n of the specific algorithm
used

Input:

The encryption key $K \in \mathcal{B}^k$

The ciphertext $C \in \mathcal{B}^c$

The nonce $\text{nonce} \in \mathcal{B}^n$

Associated data $(\text{associated}_0, \dots, \text{associated}_{d-1}) \in (\mathbb{A}_{UCS}^*)^d$, s.t. $d \in \mathbb{N}$

Require: $\text{length}(\text{StringToByteArray}(\text{associated}_i)) \leq 255 \ \forall i \triangleright$ The length is the number
of bytes in the byte array

Operation:

- 1: $\text{associated} \leftarrow \langle \rangle$ $\triangleright$ Start with the empty byte array
- 2: **for** $i \in [0, d)$ **do**
- 3: $\text{associated}_{i, \text{bytes}} \leftarrow \text{StringToByteArray}(\text{associated}_i)$ $\triangleright$ See algorithm 3.12
- 4: $\text{associated} \leftarrow \text{associated} \parallel \text{length}(\text{associated}_{i, \text{bytes}}) \parallel \text{associated}_{i, \text{bytes}}$ $\triangleright$ Since the
length is restricted to 255, it is encoded as a single unsigned byte
- 5: **end for**
- 6: $P \leftarrow \text{AuthenticatedDecryption}(K, \text{nonce}, \text{associated}, C)$ $\triangleright$ As per [24], section 2.2

Output:

The authenticated plaintext $P \in \mathcal{B}^p$

Or $\perp$ if the ciphertext does not authenticate

Test values are provided in the [get-plaintext-symmetric.json](#) file.

7 Digital Signatures

An integral part of the security of any distributed system consists of ensuring that each message did indeed originate from an authorized party.

In the context of the Swiss Post e-voting system in particular, and in systems with distribution of trust in general, this further entails requiring that each contribution comes from the expected party.

The pseudocode algorithms provided in this section rely on established digital signature standards providing authenticity and integrity of the communications. The specific algorithms used are defined in section 2. These elements are meant to address the issues raised by Thomas Haines [16].

They are meant to be used in the following way:

- the operators of each authority generate a private key and a certificate for the matching public key,
- a well-documented process ensures that each authority loads and securely stores the other authorities' certificates, validating their authenticity,
- upon sending messages, each authority signs them with their private key,
- upon receiving messages, each authority verifies that the signature is valid for the message, using the public key contained in the certificate of the purported author,
- auditors of the system are also provided with each authority's certificate, and ensure that each certificate has indeed been sent by the operators of the corresponding authority,
- finally, the auditors verify that each message has indeed been signed correctly by the expected authority.

The pseudocode algorithms below rely on well-established standards, with well-tested libraries available in most programming languages. The abstraction level in this section diverges slightly from the rest of the document, to put the focus more on the nature of the elements required rather than their exact form, therefore allowing flexibility to use existing libraries, rather than reinventing the wheel.

7.1 Generating a Signing Key and Certificate

The following algorithm is used by each authority to generate a private key and a certificate containing the corresponding public key, with use restricted to signing, for a limited duration.

The period of validity for the certificate should strike a balance between limiting the risks related to prolonged use and practicality. Since each participant (4 sets of authority operators, and typically at least one auditor per electoral authority using the system) needs to verify that each certificate is provided by the expected set of operators, the process is inherently tedious.

Given the signature and verification algorithms defined in section 2, we assume there exist matching functions for: key pair generation, which we will note `GenKeyPair()`; creation of a certificate for the public key, signed by the private key, as a self-signed x.509 v3 certificate [7] encoded according to DER [31], which we will note as follows: `GetCertificate(pubKey, privKey, info)` where the third parameter defines the additional properties of the certificate, including identity information, validity, and key usage.

Further more, the certificate should have the following properties:

- the serial number of the certificate consists of 20 random bytes,
- the basic constraints extension should be present, since this is a self-signed certificate, with the CA property set to true and the maximal chain length being restricted to 0,
- no revocation information is necessary, since the certificates management is delegated to a human-led process, whereby only the currently valid certificates should be available in the truststores, certificate revocation and rotation must be handled manually.

Algorithm 7.1 GenKeysAndCert: Generate a key pair and matching certificate

Context:

The signature algorithms, providing GenKeyPair and GetCertificate as described above

Information about the identity of the authority generating keys, including

- common name CN
- country C
- state ST
- locality L
- organisation O

Input:

Start of validity validFrom

End of validity validUntil

Require: validFrom < validUntil

Operation:

- 1: (privKey, pubKey) $\leftarrow$ GenKeyPair()
 - 2: info $\leftarrow$ {CN, C, ST, L, O}
 - 3: info $\leftarrow$ info $\cup$ {validFrom, validUntil}
 - 4: usage $\leftarrow$ (CertificateSign, DigitalSignature)
 - 5: info $\leftarrow$ info $\cup$ {usage}
 - 6: cert $\leftarrow$ GetCertificate(privKey, pubKey, info)
 - 7: **return** (privKey, cert)
-

Output:

the private key privKey which the authority will keep secret and use for signing,
the certificate cert which will be shared with the other authorities, so that they can
verify messages signed by this authority.

7.2 Importing a Trusted Certificate

We assume each authority has access to a dedicated trust store or a similar trust mechanism. These trust stores are initially empty and only properly validated certificates can be imported. The distribution of certificates must rely on an existing authenticated channel and the process for the distribution must be documented in sufficient detail. The validation of the certificate previous to the import is a human-led process, which requires (at least) the following checks:

- Does the identity claimed by the certificate match the identity of the authority? This includes validating the ASN.1 fields for country, state, locality, organisation and common name.
- Does the period of validity declared in the certificate match the expected period?
- Are the declared uses for the key exclusively restricted to 1) signing the certificate itself and 2) digital signature only?

Only after those elements have all been verified, should a certificate be imported into the authority's trust store.

7.3 Signing a Message

Given the signature algorithm defined in section 2, we note $\text{Sign}(\text{privKey}, m)$ the underlying signature algorithm, returning the byte array representing the signature. Every outgoing message from the authorities MUST be signed according to algorithm 7.2.

We assume that the current timestamp can be retrieved with $\text{GetTimestamp}()$. Further, we assume that the validity starting and ending timestamps for the certificate cert can be retrieved with $\text{ValidFrom}(\text{cert})$ and $\text{ValidUntil}(\text{cert})$ respectively.

Given the potential complexity of objects and in order to avoid relying on the interpretation of various file formats and variants (*e.g.* XML, JSON, indentation with spaces vs tabs, compact vs pretty printed files), we want to sign the content of the data, rather than its format. To that effect, we rely on the recursive hash function (see algorithm 5.5) to hash the data before signing the corresponding hash.

Algorithm 7.2 GenSignature: Generate a signature for the given message

Context:

The private key privKey
The matching certificate cert

Input:

The message to sign $m \in \mathcal{V}$
Additional context data $c \in \mathcal{V}$

Operation:

```
1:  $t \leftarrow \text{GetTimestamp}()$ 
2: if  $\text{ValidFrom}(\text{cert}) \leq t < \text{ValidUntil}(\text{cert})$  then
3:    $h \leftarrow \text{RecursiveHash}(m, c)$  ▷ See algorithm 5.5
4:   return  $\text{Sign}(\text{privKey}, h)$ 
5: else
6:   return  $\perp$ 
7: end if
```

Output:

The signature $s \in \mathcal{B}^{384}$
Or $\perp$ if the message is timestamped at a date the certificate cert is not valid for.

7.4 Verifying a Message

Given the signature algorithm defined in section 2, we note $\text{Verify}(\text{pubKey}, m, s)$ the underlying signature verification algorithm, returning $\top$ if the signature is valid, $\perp$ otherwise. Every incoming message expected to originate from a system authority **MUST** be verified according to algorithm 7.3.

As mentioned in section 7.2, we assume each authority keeps a trust store of certificates, containing only verified and validated certificates for the known authorities of the system.³ We assume each certificate cert is identified by a unique string id, and can be retrieved with $\text{FindCertificate}(\text{id})$, and the included public key can be retrieved with $\text{GetPublicKey}(\text{cert})$.

As in section 7.3, we assume that the current timestamp can be retrieved with $\text{GetTimestamp}()$. Further, we assume that the validity starting and ending timestamps for the certificate cert can be retrieved with $\text{ValidFrom}(\text{cert})$ and $\text{ValidUntil}(\text{cert})$ respectively.

Algorithm 7.3 *VerifySignature*: Verify that a signature is valid, and from the expected authority

Context:

The signature algorithms, providing Verify as described above
The trust store, providing FindCertificate as described above

Input:

The identifier of the authority expected to have signed the message $\text{id} \in \mathbb{A}_{UCS}^*$
The message to sign $m \in \mathcal{V}$
Additional context data $c \in \mathcal{V}$
The signature $s \in \mathcal{B}^{384}$

Operation:

```

cert ← FindCertificate(id)
t ← GetTimestamp()
if  $t < \text{ValidFrom}(\text{cert}) \vee t \geq \text{ValidUntil}(\text{cert})$  then                                ▷ Check validity
    return  $\perp$ 
end if
pubKey ← GetPublicKey(cert)
h ← RecursiveHash(m, c)                                                            ▷ See algorithm 5.5
return  $\text{Verify}(\text{pubKey}, h, s)$ 

```

Output:

$\top$ if the signature is valid and the message has a timestamp during which the certificate was valid, $\perp$ otherwise.

³No **web of trust**, only directly authenticated parties.

8 ElGamal Cryptosystem

The computational proof [29] describes the security properties of the ElGamal encryption scheme. Moreover, it explains that we can share the randomness when encrypting multiple messages (using different public keys). Optimizing the encryption scheme in this way is called multi-recipient ElGamal encryption and prevents us from repeatedly computing the left-hand side of the ciphertext.

8.1 Primality Testing

The generation of the encryption parameters in algorithm 8.1 needs a probabilistic primality check. We use the probabilistic Miller-Rabin [26, 30] primality test. We assume a function `MillerRabin` [25, Chapter 4.2.3] which takes as input the prime candidate to be tested and the number of iterations. This function does a probabilistic Miller-Rabin primality test with randomly chosen bases. Running the function `MillerRabin` with $\lambda/2$ iterations results in a worst case error bound of $4^{-\lambda/2} = 2^{-\lambda}$. That is, both p and q output from algorithm 8.1 are prime with a probability exceeding $1 - 2^{-\lambda}$, where λ is the security strength defined in section 2.

8.2 Parameters Generation

We instantiate the ElGamal encryption scheme over the group of nonzero quadratic residues $\mathbb{G}_q \subset \mathbb{Z}_p$, defined by the following *public* parameters: modulus p , cardinality (order) q , and generator g . We pick p and q prime with $p = 2q + 1$. Section 2 defines the bit length of p and q .

We choose g as the smallest integer $x > 1$ such that $x \in \mathbb{G}_q$. x is a generator of $\mathbb{G}_q$, like all other elements $x \in \mathbb{G}_q, x \neq 1$. If $2 \in \mathbb{G}_q$ we choose $g = 2$. If not, we choose $g = 3$. For q prime and $p = 2q + 1 \geq 11$ prime, p must be of the form $p = 12k + 11$ for some $k \geq 0$. Then $3 \in \mathbb{G}_q$ follows from the law of quadratic reciprocity.

We pick all the group parameters *verifiably* to demonstrate that they are devoid of hidden properties or back doors. Algorithm 8.1 details the verifiable selection of group parameters. The method takes a seed—the name of the election event—as an input. It leverages the SHAKE256 algorithm which produces a variable length digest [10]. Most implementations of SHAKE256 require as input a byte array and a length in bytes and the method returns a byte array. We need the output value q to be in the interval $[2^{|q|-1}, 2^{|q|}]$. To ensure this, we let the initial candidate value for q be in the interval $[2^{|q|-1}, 1.5 \cdot 2^{|q|-1}]$. This is done by prepending the bits "10", (the byte <0x02>) to the output of SHAKE256, see section 3.2 on how we represent integers, and then performing a subsequent bitwise right-shift operation to remove the last 3 bits.

For all positive integers q , we have $q = 6k + r$ for some non-negative integer k and $r \in \{0, 1, 2, 3, 4, 5\}$. We know that q is not prime for $r \in \{0, 2, 3, 4\}$ and that $p = 2q + 1$ cannot be prime for $r = 1$. Therefore, in algorithm 8.1, we let all prime candidates q be of the form $q = 6k + 5$.

Algorithm 8.1 is optimized for performance. Probabilistic primality testing as described

in section 8.1 is costly. By testing the prime candidates for compositeness using a fixed list of small prime numbers before applying more costly probabilistic primality testing, the performance can be increased. However, for prime candidates of the bit lengths as defined in table 2, the compositeness testing is also costly. Therefore, algorithm 8.1 deviates from the approach used for example in the FIPS 186-5 standard [6]. Both approaches choose uniformly random an initial candidate q . Candidate q and the corresponding $p = 2q + 1$ are tested for primality. In the approach from the FIPS 186-5 standard, when q or p is not prime, a new starting point q is chosen uniformly at random. Algorithm 8.1 on the other hand, chooses its next candidate q incrementally (by adding 6 to candidate q). The incremental search allows to test the candidates for compositeness against the list of small primes much more efficiently, by retaining the remainders of the divisions. This significantly speeds up the algorithm's performance.

The approach from the FIPS 186-5 standard results in uniformly chosen primes. The approach of algorithm 8.1 results in a higher probability of selecting a prime when it follows a wide range containing no primes. However, with the bit lengths we are operating with, as defined in table 2, a prime fixed in advance still has a negligible chance of being chosen.

Algorithm 8.1 GetEncryptionParameters

Context:

The security level defining the security strength λ , and the bit lengths of p and q , according to table 2.

Input:

$\text{seed} \in \mathbb{A}_{UCS}^*$ ▷ The name of the election event
 $\text{sp} = (5, 7, 11, \dots, \text{sp}_{l-1}), \text{sp}_i \in \mathbb{P}$ ▷ A list of small primes

Require: $|p| \bmod 8 = 0$ ▷ The algorithm assumes that the bit length of p is a multiple of 8

Operation:

```

1:  $\hat{q}_b \leftarrow \text{SHAKE256}(\text{StringToByteArray}(\text{seed}), \frac{|p|}{8})$  ▷ See algorithm 3.12 and FIPS 202 [10]
2:  $q_b \leftarrow \langle 0x02 \rangle \parallel \hat{q}_b$  ▷ Byte array concatenation
3:  $q' \leftarrow \text{ByteArrayToInteger}(q_b) \gg 3$  ▷ See algorithm 3.8 ▷ Bit-wise right shift
4:  $q \leftarrow q' - (q' \bmod 6) + 5$  ▷ Ensuring that  $q = 6k + 5$ 
5:  $r \leftarrow ()$  ▷ Make a list of the residues of  $q$  modulo the small primes
6: for  $i \in [0, l)$  do
7:    $r_i \leftarrow q \bmod \text{sp}_i$ 
8: end for
9:  $\delta \leftarrow 0$ 
10: do
11:   do
12:      $\delta \leftarrow \delta + 6$  ▷ Proceed to next prime candidate
13:      $i \leftarrow 0$  ▷ Reset the counter  $i$ 
14:     while  $i < l$  do
15:       if  $((r_i + \delta = 0 \bmod \text{sp}_i) \text{ or } (2(r_i + \delta) + 1 = 0 \bmod \text{sp}_i))$  then
16:          $\delta \leftarrow \delta + 6$  ▷ Proceed to next prime candidate
17:          $i \leftarrow 0$  ▷ Reset the counter  $i$ 
18:       else
19:          $i \leftarrow i + 1$  ▷ Proceed to next small prime of the list
20:       end if
21:     end while
22:     while  $\neg(\text{MillerRabin}(q + \delta, 1)) \text{ or } \neg(\text{MillerRabin}(2(q + \delta) + 1, 1))$ 
23:       while  $\neg(\text{MillerRabin}(q + \delta, \lambda/2)) \text{ or } \neg(\text{MillerRabin}(2(q + \delta) + 1, \lambda/2))$ 
24:          $q \leftarrow q + \delta$ 
25:          $p \leftarrow 2 \cdot q + 1$ 
26:         if  $2 \in \mathbb{G}_q$  then ▷ Choose generator of group  $\mathbb{G}_q$ 
27:            $g \leftarrow 2$ 
28:         else
29:            $g \leftarrow 3$ 
30:         end if

```

Output:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
 Group generator $g \in \mathbb{G}_q$

Test values for the algorithm 8.1 are provided in the attached [get-encryption-parameters.json](#) file.

8.3 Prime Selection

The Swiss Post Voting System encodes voting options using small primes. Given a mathematical group, we require an algorithm that returns a list of small prime numbers of this mathematical group. Algorithm 8.2 excludes 2 and 3 from the list because they are, in the Swiss Post Voting System, the possible choices for the generator g .

Algorithm 8.2 GetSmallPrimeGroupMembers

Input:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
 Group generator $g \in \mathbb{G}_q$
 Desired number of prime group members $r \in \mathbb{N}^+$

Require:

$g \in [2, 3]$
 $r \leq q - 4$
 $r < 10,000$ ▷ For efficiency reasons

Operation:

```

1: current ← 5
2: p ← ()
3: count ← 0
4: while count < r ∧ current < p do
5:   if current ∈  $\mathbb{G}_q$  ∧ IsSmallPrime(current) then ▷ See algorithm 8.3
6:     p[count] ← current
7:     count ← count + 1
8:   end if
9:   current ← current + 2
10: end while
11: if count ≠ r then
12:   return ⊥
13: end if

```

Output:

The small prime group members in ascending order $\mathbf{p} = (p_0, \dots, p_{r-1})$, $p_i \in (\mathbb{G}_q \cap \mathbb{P}) \setminus \{2, 3\}$
 ⊥ if r is bigger than the number of primes in the $\mathbb{G}_q$ group.

We define a deterministic primality test that is efficient for small primes.

Algorithm 8.3 lsSmallPrime

Context:

Input:

Number $n \in \mathbb{N}^+$

Require: $n < 2^{31}$

▷ This covers up to the 105 millionth prime

Operation:

```
1: if  $n = 1$  then
2:   return  $\perp$ 
3: else if  $n = 2$  or  $n = 3$  then
4:   return  $\top$ 
5: else if  $n \bmod 2 = 0$  or  $n \bmod 3 = 0$  then
6:   return  $\perp$ 
7: else
8:    $i \leftarrow 5$ 
9:   while  $i \leq \lceil \sqrt{n} \rceil$  do ▷ We use ceil to take into account floating point arithmetic
    limitations
10:    if  $n \bmod i = 0$  or  $n \bmod (i + 2) = 0$  then
11:      return  $\perp$ 
12:    end if
13:     $i \leftarrow i + 6$ 
14:  end while
15:  return  $\top$ 
16: end if
```

Output:

$\top$ if n is prime, $\perp$ otherwise.

8.4 Key Pair Generation

Algorithm 8.4 describes the generation of a multi-recipient ElGamal key pair. We include 0 and 1, in the secret and public key ranges respectively, since the ElGamal encryption scheme is correct and semantically secure, even for those edge cases [3, 34]. However, one must be careful when using the key pair for purposes other than ElGamal encryption, especially when an adversary might maliciously choose the key pair. For instance, imagine if one successively exponentiates a value x by the keys $k_1, \dots, k_n$:

$$(((x^{k_1})^{k_2}) \dots)^{k_n}$$

Here, a single key $k_i = 0$ would cancel the contributions of all other keys since the result is guaranteed to be 1 — potentially leading to undesired consequences. In that case, the cryptographic protocol must draw the secret keys from $\mathbb{Z}_q^+$ (to exclude 0) and the public keys from the generators of $\mathbb{G}_q$ (to exclude 1).

Algorithm 8.4 GenKeyPair: Generate a multi-recipient key pair

Input:

- Group modulus $p \in \mathbb{P}$
 - Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
 - Group generator $g \in \mathbb{G}_q$
 - Number of key elements $N \in \mathbb{N}^+$
-

Operation:

- 1: **for** $i \in [0, N)$ **do**
 - 2: $sk_i \leftarrow \text{GenRandomInteger}(q)$ ▷ See algorithm 5.1
 - 3: $pk_i \leftarrow g^{sk_i} \bmod p$
 - 4: **end for**
-

Output:

A pair of secret and public keys $(\mathbf{sk}, \mathbf{pk}), \mathbf{sk} \in \mathbb{Z}_q^N, \mathbf{pk} \in \mathbb{G}_q^N$

8.5 Encryption

We consider a “multi-recipient message”, in which different public keys encrypt different messages. If there are more public keys than messages ($\ell < k$), we drop the excess public keys.

Algorithm 8.5 GetCiphertext: Compute a ciphertext with provided randomness

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
Group generator $g \in \mathbb{G}_q$

Input:

A multi-recipient message $\mathbf{m} \in \mathbb{G}_q^\ell$
The random exponent to use $r \in \mathbb{Z}_q$
A multi-recipient public key $\mathbf{pk} \in \mathbb{G}_q^k$

Require: $0 < \ell \leq k$

Operation:

```
1:  $\gamma \leftarrow g^r \bmod p$ 
2: for  $i \in [0, \ell)$  do
3:    $\phi_i \leftarrow \mathbf{pk}_i^r \cdot m_i \bmod p$ 
4: end for
```

Output:

The ciphertext $(\gamma, \phi_0, \dots, \phi_{\ell-1}) \in \mathbb{H}_\ell$

Test values for the algorithm 8.5 are provided in [get-ciphertext.json](#).

8.6 Ciphertext Operations

Algorithm 8.6 GetCiphertextExponentiation: Exponentiate each ciphertext element by an exponent a

Context:

- Group modulus $p \in \mathbb{P}$
- Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
- Group generator $g \in \mathbb{G}_q$

Input:

- A multi-recipient ciphertext $C = (\gamma, \phi_0, \dots, \phi_{\ell-1}) \in \mathbb{H}_\ell$
 - An exponent $a \in \mathbb{Z}_q$
-

Operation:

- 1: $\gamma \leftarrow \gamma^a \bmod p$
 - 2: **for** $i \in [0, \ell)$ **do**
 - 3: $\phi_i \leftarrow \phi_i^a \bmod p$
 - 4: **end for**
-

Output:

- $(\gamma, \phi_0, \dots, \phi_{\ell-1}) \in \mathbb{H}_\ell$
-

Algorithm 8.7 GetCiphertextVectorExponentiation: Exponentiate a vector of ciphertexts and take the product

Context:

- Group modulus $p \in \mathbb{P}$
- Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
- Group generator $g \in \mathbb{G}_q$

Input:

- A vector of ciphertexts $\vec{C} = (C_0, \dots, C_{n-1}) \in \mathbb{H}_\ell^n$
 - A vector of exponents $\vec{a} = (a_0, \dots, a_{n-1}) \in \mathbb{Z}_q^n$
-

Operation:

- 1: $\text{product} \leftarrow \underbrace{(1, \dots, 1)}_{\ell+1 \text{ times}} \quad \triangleright \text{Neutral element of ciphertext multiplication}$
 - 2: **for** $i \in [0, n)$ **do**
 - 3: $\text{product} \leftarrow \text{GetCiphertextProduct}(\text{product}, \text{GetCiphertextExponentiation}(C_i, a_i))$
 $\triangleright$ See algorithm 8.8 and algorithm 8.6
 - 4: **end for**
-

Output:

- The resulting product $\in \mathbb{H}_\ell$
-

Algorithm 8.8 GetCiphertextProduct: Multiply two ciphertexts

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
Group generator $g \in \mathbb{G}_q$

Input:

A multi-recipient ciphertext $C_a = (\gamma_a, \phi_{a,0}, \dots, \phi_{a,\ell-1}) \in \mathbb{H}_\ell$
Another multi-recipient ciphertext $C_b = (\gamma_b, \phi_{b,0}, \dots, \phi_{b,\ell-1}) \in \mathbb{H}_\ell$

Operation:

```
1:  $\gamma \leftarrow \gamma_a \cdot \gamma_b \pmod{p}$ 
2: for  $i \in [0, \ell)$  do
3:    $\phi_i \leftarrow \phi_{a,i} \cdot \phi_{b,i} \pmod{p}$ 
4: end for
```

Output:

$(\gamma, \phi_0, \dots, \phi_{\ell-1}) \in \mathbb{H}_\ell$

Test values for the algorithm 8.8 are provided in [get-ciphertext-product.json](#).

8.7 Decryption

Algorithm 8.9 GetMessage: Retrieve the message from a ciphertext

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
 Group generator $g \in \mathbb{G}_q$

Input:

A multi-recipient ciphertext $\mathbf{c} = (\gamma, \phi_0, \dots, \phi_{\ell-1}) \in \mathbb{H}_\ell$
 A multi-recipient secret key $\mathbf{sk} \in \mathbb{Z}_q^k, 0 < \ell \leq k$

Operation:

```

1: for  $i \in [0, \ell)$  do
2:    $m_i \leftarrow \phi_i \cdot \gamma^{-\mathbf{sk}_i} \bmod p$ 
3: end for
    
```

Output:

The multi-recipient message $(m_0, \dots, m_{\ell-1}) \in \mathbb{G}_q^\ell$

Since the system uses a multi-party re-encryption/decryption mixnet, in which the combined public key of the parties is used for encryption, each party actually performs a partial decryption. This entails that the actual output of the decryption phase for each party is actually still a ciphertext and the value for γ needs to be preserved to allow decryption by the following parties. This gives us the partial decryption algorithm in algorithm 8.10.

Algorithm 8.10 GetPartialDecryption: Partially decrypt a provided ciphertext

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
 Group generator $g \in \mathbb{G}_q$

Input:

A multi-recipient ciphertext $\mathbf{c} = (\gamma, \phi_0, \dots, \phi_{\ell-1}) \in \mathbb{H}_\ell$
 A multi-recipient secret key $\mathbf{sk} \in \mathbb{Z}_q^k, 0 < \ell \leq k$

Operation:

```

1:  $(m_0, \dots, m_{\ell-1}) \leftarrow \text{GetMessage}(\mathbf{c}, \mathbf{sk})$  ▷ See algorithm 8.9
2: return  $(\gamma, m_0, \dots, m_{\ell-1})$ 
    
```

Output:

The multi-recipient ciphertext $(\gamma, m_0, \dots, m_{\ell-1}) \in \mathbb{H}_\ell$

Algorithm 8.11 GenVerifiableDecryptions: Provide a verifiable partial decryption of a vector of ciphertexts.

Context:

- Group modulus $p \in \mathbb{P}$
- Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
- Group generator $g \in \mathbb{G}_q$

Input:

- A vector of ciphertexts $\mathbf{C} = (\mathbf{c}_0, \dots, \mathbf{c}_{N-1}) \in \mathbb{H}_\ell^N$
- A multi-recipient key pair $(\mathbf{pk}, \mathbf{sk}) \in \mathbb{G}_q^k \times \mathbb{Z}_q^k$
- An array of optional additional information $\mathbf{i}_{\text{aux}} \in (\mathbb{A}_{UCS^*})^s, s \in \mathbb{N}$

Require: $N > 0$

Require: $0 < \ell \leq k$

Operation:

- 1: **for** $i \in [0, N)$ **do**
 - 2: $\mathbf{c}'_i \leftarrow \text{GetPartialDecryption}(\mathbf{c}_i, \mathbf{sk})$ ▷ See algorithm 8.10
 - 3: $(\gamma', \phi'_0, \dots, \phi'_{\ell-1}) \leftarrow \mathbf{c}'_i$
 - 4: $\pi_{\text{dec},i} \leftarrow \text{GenDecryptionProof}(\mathbf{c}_i, (\mathbf{pk}, \mathbf{sk}), (\phi'_0, \dots, \phi'_{\ell-1}), \mathbf{i}_{\text{aux}})$ ▷ See algorithm 10.5
 - 5: **end for**
 - 6: $\mathbf{C}' \leftarrow (\mathbf{c}'_0, \dots, \mathbf{c}'_{N-1})$
 - 7: $\pi_{\text{dec}} \leftarrow (\pi_{\text{dec},0}, \dots, \pi_{\text{dec},N-1})$
 - 8: **return** $(\mathbf{C}', \pi_{\text{dec}})$
-

Output:

- A vector of partially decrypted ciphertexts $\mathbf{C}' = (\mathbf{c}'_0, \dots, \mathbf{c}'_{N-1}) \in \mathbb{H}_\ell^N$
 - A vector of decryption proofs $\pi_{\text{dec}} \leftarrow (\pi_{\text{dec},0}, \dots, \pi_{\text{dec},N-1}) \in (\mathbb{Z}_q \times \mathbb{Z}_q^\ell)^N$
-

Algorithm 8.12 VerifyDecryptions: Verify the decryptions of a vector of ciphertexts.

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
 Group generator $g \in \mathbb{G}_q$

Input:

A vector of ciphertexts $\mathbf{C} = (\mathbf{c}_0, \dots, \mathbf{c}_{N-1}) \in \mathbb{H}_\ell^N$
 A multi-recipient public key $\mathbf{pk} \in \mathbb{G}_q^k$
 A vector of partially decrypted ciphertexts $\mathbf{C}' = (\mathbf{c}'_0, \dots, \mathbf{c}'_{N-1}) \in \mathbb{H}_\ell^N$
 A vector of decryption proofs $\pi_{\text{dec}} = (\pi_{\text{dec},0}, \dots, \pi_{\text{dec},N-1}) \in (\mathbb{Z}_q \times \mathbb{Z}_q^\ell)^N$
 An array of optional additional information $\mathbf{i}_{\text{aux}} \in (\mathbb{A}_{UCS}^*)^s, s \in \mathbb{N}$

Require: $N > 0$

Require: $0 < \ell \leq k$

Operation:

```

1: for  $i \in [0, N)$  do
2:    $(\gamma, \phi_0, \dots, \phi_{\ell-1}) \leftarrow \mathbf{c}_i$ 
3:    $(\gamma', \phi'_0, \dots, \phi'_{\ell-1}) \leftarrow \mathbf{c}'_i$ 
4:   if  $\gamma \neq \gamma'$  then
5:     return  $\perp$ 
6:   end if
7:    $\mathbf{m} \leftarrow (\phi'_0, \dots, \phi'_{\ell-1})$ 
8:    $\text{ok} \leftarrow \text{VerifyDecryption}(\mathbf{c}_i, \mathbf{pk}, \mathbf{m}, \pi_{\text{dec},i}, \mathbf{i}_{\text{aux}})$  ▷ See algorithm 10.6
9:   if  $\neg \text{ok}$  then
10:    return  $\perp$ 
11:  end if
12: end for
13: return  $\top$  ▷ Succeed if and only if all verifications above succeeded

```

Output:

The result of the verification: $\top$ if **all** the verifications are successful, $\perp$ otherwise.

8.8 Combining ElGamal Multi-Recipient Public Keys

In this section we provide a mechanism to combine public keys in such a way that encryption under the combined public key can be inverted by successive decryptions with the secret key components. This allows consumers to encrypt their messages in a way that requires the collaboration of all authorities to retrieve the plaintext, without those authorities ever needing to reveal their secret key. Such trust distribution systems can be subjected to rogue key attacks, where a component's public key is built using parts of the other components' public keys, leading to possible attacks on the combined key. For this reason, systems relying on this mechanism to distribute trust should consider using zero-knowledge proofs of knowledge of the corresponding secret key.

Since we are using ElGamal encryption, this can be achieved by simply taking the product of the public key components as the combined key.

By construction of the public keys, we have $\mathbf{pk}_j = g^{\mathbf{sk}_j}$. It follows that $\prod_j \mathbf{pk}_j = g^{\sum_j \mathbf{sk}_j}$. By definition of the ElGamal encryption function, we have $E_{\prod_j \mathbf{pk}_j}(m, r) = (g^r, (\prod_j \mathbf{pk}_j)^r \cdot m)$. Using the previous equality to replace the product of public keys, we get $(g^r, g^{r \cdot \sum_j \mathbf{sk}_j} \cdot m)$. From this point, each successive partial decryption by the corresponding secret key $\mathbf{sk}_j$ multiplies the second term by $g^{r \cdot (-\mathbf{sk}_j)}$, eventually leaving only the term m . This can be generalised for the multi-recipient ElGamal encryption scheme, by combining each set of keys independently.

Algorithm 8.13 CombinePublicKeys: Combine a set of multi-recipient ElGamal keys

Context:

Group modulus $p \in \mathbb{P}$

Input:

A list of multi-recipient ElGamal public keys $(\mathbf{pk}_0, \dots, \mathbf{pk}_{s-1}) \in \mathbb{G}_q^{s \times N}$

▷ Where s is the number of keys, and N is the number of elements in each multi-recipient key

Require: $s > 0$

Require: $N > 0$

Operation:

- 1: **for** $i \in [0, N)$ **do**
 - 2: $\mathbf{pk}_{\text{combined}, i} \leftarrow \prod_{j=0}^{s-1} \mathbf{pk}_{j, i} \bmod p$
 - 3: **end for**
 - 4: $\mathbf{pk}_{\text{combined}} = (\mathbf{pk}_{\text{combined}, 0}, \dots, \mathbf{pk}_{\text{combined}, N-1})$
-

Output:

$\mathbf{pk}_{\text{combined}} \in \mathbb{G}_q^N$

9 Mix Net

Verifiable mix nets underpin most modern e-voting schemes with non-trivial tallying methods since they hide the relationship between encrypted votes (potentially linked to the voter's identifier) and decrypted votes [17]. A re-encryption mix net consists of a sequence of mixers, each of which shuffles and re-encrypts an input ciphertext list and returns a different ciphertext list containing the same plaintexts. Each mixer proves knowledge of the permutation and the randomness (without revealing them to the verifier). Verifying these proofs guarantees that no mixer added, deleted, or modified a vote. The most widely used verifiable mix nets are the ones from Terelius-Wikström [33] and Bayer-Groth [2]. The Swiss Post Voting System uses the Bayer-Groth mix net, which we describe in this section. The computational proof [29] discusses the security properties of the non-interactive version of the Bayer-Groth mix net. Please note that each control component in the Swiss Post Voting System combines a verifiable shuffle with a subsequent, verifiable decryption step. This section details only the verifiable shuffle. Our implementation exposes the following two public methods:

Algorithm 9.1 GenVerifiableShuffle: Shuffle (including re-encryption), and provide a Bayer-Groth proof of the shuffle

Context:

- Group modulus $p \in \mathbb{P}$
- Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
- Group generator $g \in \mathbb{G}_q$

Input:

- A vector of ciphertexts $\mathbf{C} \in \mathbb{H}_\ell^N$
- A multi-recipient public key $\mathbf{pk} \in \mathbb{G}_q^k$ $\triangleright$ This public key is passed as context to all sub-arguments

Require: $0 < \ell \leq k$

Require: $2 \leq N \leq q - 3$

Operation:

- 1: $(\mathbf{C}', \pi, \mathbf{r}) \leftarrow \text{GenShuffle}(\mathbf{C}, \mathbf{pk})$ $\triangleright$ See algorithm 9.3
 - 2: $(m, n) \leftarrow \text{GetMatrixDimensions}(N)$ $\triangleright$ See algorithm 9.5
 - 3: $\mathbf{ck} \leftarrow \text{GetVerifiableCommitmentKey}(n)$ $\triangleright$ See algorithm 9.6 $\triangleright$ This commitment key is passed as context to all sub-arguments
 - 4: $\text{shuffleStatement} \leftarrow (\mathbf{C}, \mathbf{C}')$
 - 5: $\text{shuffleWitness} \leftarrow (\pi, \mathbf{r})$
 - 6: $\text{shuffleArgument} \leftarrow \text{GetShuffleArgument}(\text{shuffleStatement}, \text{shuffleWitness}, m, n)$ $\triangleright$ See algorithm 9.11
-

Output:

- $\mathbf{C}' \in \mathbb{H}_\ell^N$
 - shuffleArgument $\triangleright$ See algorithm 9.11 for the domain
-

Algorithm 9.2 VerifyShuffle: Verify the output of a previously generated verifiable shuffle

Context:

- Group modulus $p \in \mathbb{P}$
- Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
- Group generator $g \in \mathbb{G}_q$

Input:

- A vector of unshuffled ciphertexts $\mathbf{C} \in \mathbb{H}_\ell^N$
- A vector of shuffled, re-encrypted ciphertexts $\mathbf{C}' \in \mathbb{H}_\ell^N$
- A Bayer-Groth shuffleArgument ▷ See algorithm 9.11 for the domain
- A multi-recipient public key $\mathbf{pk} \in \mathbb{G}_q^k$

Require: $0 < \ell \leq k$

Require: $2 \leq N \leq q - 3$

Operation:

- 1: $(m, n) \leftarrow \text{GetMatrixDimensions}(N)$ ▷ See algorithm 9.5
- 2: $\mathbf{ck} \leftarrow \text{GetVerifiableCommitmentKey}(n)$ ▷ See algorithm 9.6
- 3: $\text{shuffleStatement} \leftarrow (\mathbf{C}, \mathbf{C}')$
- 4: **return** $\text{VerifyShuffleArgument}(\text{shuffleStatement}, \text{shuffleArgument}, m, n)$
▷ See algorithm 9.12 ▷ $\mathbf{pk}$ and $\mathbf{ck}$ are passed as context

Output:

The result of the verification: $\top$ if the verification is successful, $\perp$ otherwise.

9.1 Pre-Requisites

9.1.1 Shuffle

Algorithm 9.3 shuffles a list of ciphertexts. We require the shuffled list of ciphertexts, the permutation, and the list of random exponents to prove the shuffle's correctness.

Algorithm 9.3 GenShuffle: Re-encrypting shuffle

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
Group generator $g \in \mathbb{G}_q$

Input:

A vector of ciphertexts $\mathbf{C} \in \mathbb{H}_\ell^N$
A multi-recipient public key $\mathbf{pk} \in \mathbb{G}_q^k$

Require: $0 < \ell \leq k$

Operation:

- 1: $(\pi_0, \dots, \pi_{N-1}) \leftarrow \text{GenPermutation}(N)$ ▷ See algorithm 9.4
- 2: **for** $i \in [0, N)$ **do**
- 3: $r_i \leftarrow \text{GenRandomInteger}(q)$ ▷ See algorithm 5.1
- 4: $e \leftarrow \text{GetCiphertext}(\vec{1}, r_i, \mathbf{pk})$ ▷ Ciphertext of a vector of ℓ 1s. See algorithm 8.5
- 5: $\mathbf{C}'_i \leftarrow \text{GetCiphertextProduct}(e, \mathbf{C}_{\pi_i})$ ▷ See algorithm 8.8
- 6: **end for**

Output:

$\mathbf{C}' = (\mathbf{C}'_0, \dots, \mathbf{C}'_{N-1}) \in \mathbb{H}_\ell^N$ ▷ The result of the shuffle
 $\pi \in \Sigma_N$ ▷ The permutation used
 $\mathbf{r} = (r_0, \dots, r_{N-1}) \in \mathbb{Z}_q^N$ ▷ The exponents used for re-encryption

Algorithm 9.4 provides a way to generate a random permutation of indices for a list of size N . It uses the algorithm formalized by Knuth [21]. The pseudocode below assumes 0-based indexing, and as such deviates from standard mathematical notation in favor of closer proximity to the implementation.

Algorithm 9.4 GenPermutation: Permutation of indices up to N

Input:

Permutation size $N \in \mathbb{N}^+$

Operation:

```

1:  $\pi \leftarrow (0, \dots, N - 1)$ 
2: for  $i \in [0, N)$  do
3:    $\text{offset} \leftarrow \text{GenRandomInteger}(N - i)$  ▷ See algorithm 5.1
4:    $\text{tmp} \leftarrow \pi_i$ 
5:    $\pi_i \leftarrow \pi_{i+\text{offset}}$ 
6:    $\pi_{i+\text{offset}} \leftarrow \text{tmp}$ 
7: end for
```

Output:

π ▷ A permutation of the values between 0 and $N - 1$

Ensure: $\forall j \in [0, N) : j \in \pi$

Ensure: $\pi \in \mathbb{Z}_N^N$ ▷ Those two elements combined ensure that $\pi \in \Sigma_N$

9.1.2 Matrix Dimensions

The Bayer-Groth mix net is memory optimal, when the ciphertexts can be arranged into matrices with an equal number of rows and columns. In the worst case, when the number of ciphertexts is prime, the resulting matrix has dimensions $1 \times N$. The below algorithm yields the optimal matrix size for a given number of ciphertexts. As an example, $N = 12$ results in $m = 3, n = 4$, $N = 18$ results in $m = 3, n = 6$, and $N = 23$ results in $m = 1, n = 23$,

Algorithm 9.5 GetMatrixDimensions: Return the optimal dimensions for the ciphertext matrix

Input:

Number of ciphertexts $N \in \mathbb{N}^+ \setminus \{1\}$

Operation:

```
1:  $m \leftarrow 1$ 
2:  $n \leftarrow N$ 
3: for  $i \in [\lfloor \sqrt{N} \rfloor, 1)$  do
4:   if  $i \mid N$  then
5:      $m \leftarrow i$ 
6:      $n \leftarrow \frac{N}{i}$ 
7:     return  $m, n$ 
8:   end if
9: end for
```

Output:

$m \in \mathbb{N}^+$
 $n \in \mathbb{N}^+ \setminus \{1\}$

9.2 Commitments

A cryptographic commitment allows a party to commit to a value (the opening), to keep the opening hidden from others, and to reveal it later [13].

We use the Pedersen commitment scheme [28] with a commitment key $\mathbf{ck} = (h, g_1, \dots, g_\nu)$ that was generated in a verifiable manner.

The Pedersen commitment scheme satisfies three properties that the Bayer-Groth mix net requires [2].

- *Perfectly hiding*: The commitment is uniformly distributed in $\mathbb{G}_q$.
- *Computationally binding*: It is computationally infeasible to find two different values producing the same commitment.
- *Homomorphic*: It holds that $\text{GetCommitment}(a + b; r + s) = \text{GetCommitment}(a; r) \text{GetCommitment}(b; s)$ for messages a, b , a commitment key $\mathbf{ck}$ and random values r, s .

The Pedersen commitment scheme is *computationally binding* only if the commitment keys are generated independently and verifiably at random:

Algorithm 9.6 GetVerifiableCommitmentKey: Generates a verifiable commitment key

Context:

Group modulus $p \in \mathbb{P}$

Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$

Input:

The desired number of elements of the commitment key $\nu \in \mathbb{N}^+$

Require: $\nu \leq q - 3$

Operation:

```

1:  $count = 0$ 
2:  $i = 0$ 
3:  $v \leftarrow \{\}$ 
4: while  $count \leq \nu$  do
5:    $u \leftarrow \text{RecursiveHashToZq}(q, (\text{"commitmentKey"}, i, count)) + 1$   $\triangleright$  See algorithm 5.6
6:    $w \leftarrow u^2 \bmod p$ 
7:   if  $w \notin \{1, g\} \cup v$  then
8:      $g_{count} \leftarrow w$ 
9:      $v \leftarrow v \cup \{g_{count}\}$ 
10:     $count = count + 1$ 
11:   end if
12:    $i = i + 1$ 
13: end while
14:  $h \leftarrow g_0$ 

```

$\triangleright$ By convention, we designate g_0 as h

Output:

A commitment key $\mathbf{ck} = (h, g_1, \dots, g_\nu) \in \mathbb{C}_\nu$

Test values for the algorithm 9.6 are provided in the attached [get-verifiable-commitment-key.json](#) file.

Algorithm 9.7 GetCommitment: Computes a commitment to a value

Context:

Group modulus $p \in \mathbb{P}$

Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$

Input:

The values to commit to $\mathbf{a} \in \mathbb{Z}_q^\ell$

A random value $r \in \mathbb{Z}_q$

A commitment key $\mathbf{ck} = (h, g_1, \dots, g_\nu) \in \mathbb{C}_\nu$ s.t. $\nu \geq \ell$

Require: $\ell > 0$

Operation:

1: $c \leftarrow h^r \prod_{i=1}^{\ell} g_i^{a_{i-1}} \bmod p$

Output:

The commitment $c \in \mathbb{G}_q$

Algorithm 9.8 GetCommitmentMatrix: Computes the commitment for a matrix

Context:

Group modulus $p \in \mathbb{P}$

Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$

Input:

The values to be committed $A \in \mathbb{Z}_q^{n \times m} \triangleright$ We note the columns of A as $\vec{a}_0, \dots, \vec{a}_{m-1}$

The random values to use $\mathbf{r} \in \mathbb{Z}_q^m$

A commitment key $\mathbf{ck} = (h, g_1, \dots, g_\nu) \in \mathbb{C}_\nu$ s.t. $\nu \geq n$

Require: $m, n > 0$

Operation:

1: **for** $i \in [0, m)$ **do**

2: $c_i \leftarrow \text{GetCommitment}(\vec{a}_i, r_i, \mathbf{ck})$

$\triangleright$ See algorithm 9.7

3: **end for**

Output:

The commitments $(c_0, \dots, c_{m-1}) \in \mathbb{G}_q^m$

This algorithm is consistent with the notation defined in [2], in section 2.3 Homomorphic Encryption:

For a matrix $A \in \mathbb{Z}_q^{n \times m}$ with columns $\vec{a}_1, \dots, \vec{a}_m$ we shorten notation by defining $\text{com}_{\mathbf{ck}}(A; \vec{r}) = (\text{com}_{\mathbf{ck}}(\vec{a}_1; r_1), \dots, \text{com}_{\mathbf{ck}}(\vec{a}_m; r_m))$

Algorithm 9.9 GetCommitmentVector: Compute the commitment for a transposed vector. This is only used in the algorithm 9.22, hence the specific indices

Context:

Group modulus $p \in \mathbb{P}$

Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$

Input:

The values to be committed $\mathbf{d} = (d_0, \dots, d_{2 \cdot m}) \in \mathbb{Z}_q^{2 \cdot m+1}$

The random values to use $\mathbf{t} = (t_0, \dots, t_{2 \cdot m}) \in \mathbb{Z}_q^{2 \cdot m+1}$

A commitment key $\mathbf{ck} = (h, g_1, \dots, g_\nu) \in \mathbb{C}_\nu$ s.t. $\nu \geq 1$

Operation:

- 1: $(c_0, \dots, c_{2 \cdot m}) \leftarrow \text{GetCommitmentMatrix}(\mathbf{d}, \mathbf{t}, \mathbf{ck})$ $\triangleright$ See algorithm 9.8, with a single row of $2 \cdot m + 1$ columns
 - 2: **return** $(c_0, \dots, c_{2 \cdot m})$
-

Output:

The commitments $(c_0, \dots, c_{2 \cdot m}) \in \mathbb{G}_q^{2 \cdot m+1}$

9.3 Arguments

Conceptually, the Bayer-Groth proof of a shuffle consists of six arguments. Figure 1 highlights the hierarchy of these arguments.

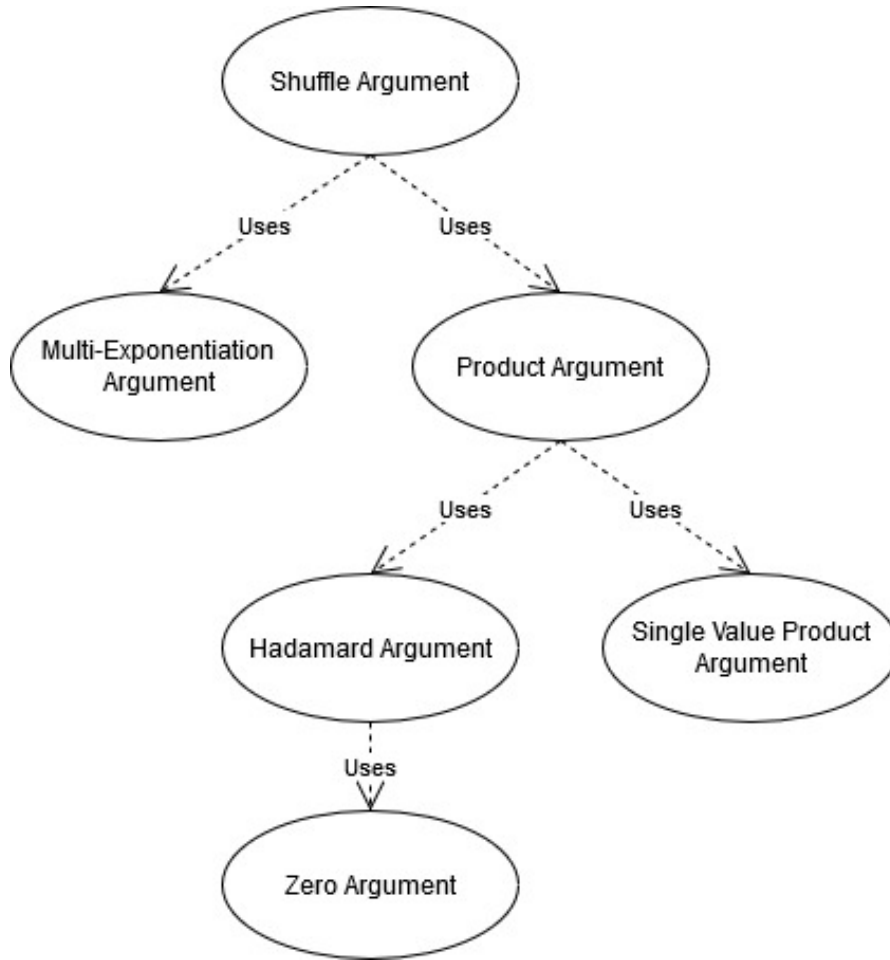


Figure 1: Bayer-Groth argument for the correctness of a shuffle

The shuffle argument invokes a multi-exponentiation and a product argument. The product argument, in turn, uses a Hadamard and a single value product argument. Finally, the Hadamard argument calls a zero argument.

In all knowledge arguments, we will use the following terminology:

statement the public information for which we assert that a property holds

witness the private information we use to make arguments on the validity of the statement

argument the information we provide to a third party which allows them to verify the validity of our statement

We prove and verify the Bayer-Groth mix net in the *non-interactive* setting: the prover and the verifier recursively hash various elements to generate the argument's challenge messages. Since the mathematical group's rank q is much larger than the output domain, converting the hash function's output to an integer of byte size L is sound ($\text{ByteLength}(q) \gg L$).

In some algorithms, we will use the bilinear algorithm $\star : \mathbb{Z}_q^n \times \mathbb{Z}_q^n \rightarrow \mathbb{Z}_q$ defined by the value y as:

$$(a_0, \dots, a_{n-1}) \star (b_0, \dots, b_{n-1}) = \sum_{j=0}^{n-1} a_j \cdot b_j \cdot y^{j+1}$$

Where all multiplications are performed modulo q .

Let us formalize it with the following pseudocode algorithm.

Algorithm 9.10 StarMap: Defines the $\star$ bilinear map

Context:

Group modulus $p \in \mathbb{P}$

Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$

Input:

Value $y \in \mathbb{Z}_q$

First vector $\mathbf{a} = (a_0, \dots, a_{n-1}) \in \mathbb{Z}_q^n$

Second vector $\mathbf{b} = (b_0, \dots, b_{n-1}) \in \mathbb{Z}_q^n$

Operation:

1: $s \leftarrow 0$

2: **for** $j \in [0, n)$ **do**

3: $s \leftarrow s + a_j \cdot b_j \cdot y^{j+1}$

4: **end for**

▷ All operations are performed modulo q

Output:

$s \in \mathbb{Z}_q$

Test values for the algorithm 9.10 are provided in the attached [bilinearMap.json](#) file.

9.3.1 Shuffle Argument

In the following pseudocode algorithm, we will generate an argument of knowledge of a permutation $\pi \in \Sigma_N$ and randomness $\rho \in \mathbb{Z}_q^N$ such that for given ciphertexts $\vec{C} \in \mathbb{H}_\ell^N$ and $\vec{C}' \in \mathbb{H}_\ell^N$ it holds that for all $i \in [0, N)$:

$$\vec{C}'_i = \text{GetCiphertextProduct}(\text{GetCiphertext}(\vec{1}, \rho_i, \mathbf{pk}), \vec{C}_{\pi(i)})$$

Algorithm 9.11 GetShuffleArgument: Compute a cryptographic argument for the validity of the shuffle

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
 Group generator $g \in \mathbb{G}_q$
 A multi-recipient public key $\mathbf{pk} \in \mathbb{G}_q^k$
 A commitment key $\mathbf{ck} = (h, g_1, \dots, g_\nu) \in \mathbb{C}_\nu$

Input:

The statement composed of
 - The incoming list of ciphertexts $\vec{C} \in \mathbb{H}_\ell^N$ s.t. $0 < \ell \leq k$
 - The shuffled and re-encrypted list of ciphertexts $\vec{C}' \in \mathbb{H}_\ell^N$
 The witness composed of
 - permutation $\pi \in \Sigma_N$
 - randomness $\vec{\rho} \in \mathbb{Z}_q^N$

The number of rows to use for ciphertext matrices $m \in \mathbb{N}^+$

The number of columns to use for ciphertext matrices $n \in \mathbb{N}^+$ s.t. $2 \leq n \leq \nu$

Require: $\forall i \in [0, N) : \vec{C}'_i = \text{GetCiphertextProduct}(\text{GetCiphertext}(\vec{C}, \rho_i, \mathbf{pk}), \vec{C}_{\pi(i)})$

Require: $N = mn$

Operation:

```

1:  $\mathbf{r} \leftarrow \text{GenRandomVector}(q, m)$  ▷ See algorithm 5.2
2:  $A \leftarrow \text{Transpose}(\text{ToMatrix}(\{\pi(i)\}_{i=0}^{N-1}, m, n))$  ▷ Create a  $n \times m$  matrix. See algorithm 9.14 and algorithm 9.13
3:  $\mathbf{c}_A \leftarrow \text{GetCommitmentMatrix}(A, \mathbf{r}, \mathbf{ck})$  ▷ See algorithm 9.8
4:  $x \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}(p, q, \mathbf{pk}, \mathbf{ck}, \vec{C}, \vec{C}', \mathbf{c}_A))$  ▷ See algorithm 3.8 and algorithm 5.5
5:  $\mathbf{s} \leftarrow \text{GenRandomVector}(q, m)$ 
6:  $\mathbf{b} \leftarrow \{x^{\pi(i)}\}_{i=0}^{N-1}$ 
7:  $B \leftarrow \text{Transpose}(\text{ToMatrix}(\mathbf{b}, m, n))$ 
8:  $\mathbf{c}_B \leftarrow \text{GetCommitmentMatrix}(B, \mathbf{s}, \mathbf{ck})$ 
9:  $y \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}(\mathbf{c}_B, p, q, \mathbf{pk}, \mathbf{ck}, \vec{C}, \vec{C}', \mathbf{c}_A))$ 
10:  $z \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}("1", \mathbf{c}_B, p, q, \mathbf{pk}, \mathbf{ck}, \vec{C}, \vec{C}', \mathbf{c}_A))$  ▷ Both  $\vec{C}$  and  $\vec{C}'$  are passed in the vector forms here
11:  $\text{Zneg} \leftarrow \text{Transpose}(\text{ToMatrix}(\{-z\}_{i=1}^N, m, n))$  ▷ Vector of length  $N$ , with all values being  $q - z$ 
12:  $\mathbf{c}_{-z} \leftarrow \text{GetCommitmentMatrix}(\text{Zneg}, \vec{0}, \mathbf{ck})$  ▷ A vector of length  $m$ , with all 0 values
13:  $\mathbf{c}_D \leftarrow \mathbf{c}_A^y \mathbf{c}_B$  ▷ Entry-wise product
14:  $D \leftarrow yA + B$ 
15:  $\mathbf{t} \leftarrow y\mathbf{r} + \mathbf{s}$ 
16:  $b \leftarrow \prod_{i=0}^{N-1} (y_i + x^i - z)$ 
17:  $\mathbf{pStatement} \leftarrow (\mathbf{c}_D \mathbf{c}_{-z}, b)$ 
18:  $\mathbf{pWitness} \leftarrow (D + \text{Zneg}, \mathbf{t})$ 
19:  $\text{productArgument} \leftarrow \text{GetProductArgument}(\mathbf{pStatement}, \mathbf{pWitness})$  ▷ See algorithm 9.18
20:  $\rho \leftarrow q - (\vec{\rho} \cdot \mathbf{b})$  ▷ Standard inner product  $\sum_{i=0}^{N-1} \rho_i b_i$ 
21:  $\vec{x} \leftarrow \{x^i\}_{i=0}^{N-1}$ 
22:  $C \leftarrow \text{GetCiphertextVectorExponentiation}(\vec{C}, \vec{x})$  ▷ See algorithm 8.7
23:  $\mathbf{mStatement} \leftarrow (\text{ToMatrix}(\vec{C}', m, n), C, \mathbf{c}_B)$  ▷ See algorithm 9.13
24:  $\mathbf{mWitness} \leftarrow (B, \mathbf{s}, \rho)$ 
25:  $\text{multiExponentiationArgument} \leftarrow \text{GetMultiExponentiationArgument}(\mathbf{mStatement}, \mathbf{mWitness})$  ▷ See algorithm 9.15

```

Output:

$\text{shuffleArgument} = (\mathbf{c}_A, \mathbf{c}_B, \text{productArgument}, \text{multiExponentiationArgument})$
 ▷ If $m = 1$ then $\text{shuffleArgument} \in \mathbb{G}_q^m \times \mathbb{G}_q^m \times (\mathbb{G}_q^3 \times \mathbb{Z}_q^{2n+2}) \times (\mathbb{G}_q^{2m+1} \times \mathbb{H}_\ell^{2m} \times \mathbb{Z}_q^{n+4})$
 ▷ If $m > 1$ then $\text{shuffleArgument} \in \mathbb{G}_q^m \times \mathbb{G}_q^m \times (\mathbb{G}_q^{3m+7} \times \mathbb{Z}_q^{4n+5}) \times (\mathbb{G}_q^{2m+1} \times \mathbb{H}_\ell^{2m} \times \mathbb{Z}_q^{n+4})$

In the following pseudocode algorithm, we verify that a provided Shuffle argument adequately supports the corresponding statement.

Algorithm 9.12 VerifyShuffleArgument: Verify a cryptographic argument for the validity of the shuffle

Context:

- Group modulus $p \in \mathbb{P}$
- Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
- Group generator $g \in \mathbb{G}_q$
- A multi-recipient public key $\mathbf{pk} \in \mathbb{G}_q^k$
- A commitment key $\mathbf{ck} = (h, g_1, \dots, g_\nu) \in \mathbb{C}_\nu$

Input:

- The **statement** composed of
 - The incoming list of ciphertexts $\vec{C} \in \mathbb{H}_\ell^N$ s.t. $0 < \ell \leq k$
 - The shuffled and re-encrypted list of ciphertexts $\vec{C}' \in \mathbb{H}_\ell^N$
- The **argument** composed of
 - the commitment vector $\mathbf{c}_A \in \mathbb{G}_q^m$
 - the commitment vector $\mathbf{c}_B \in \mathbb{G}_q^m$
 - a **productArgument** ▷ See algorithm 9.18
 - a **multiExponentiationArgument** ▷ See algorithm 9.15
- The number of rows to use for ciphertext matrices $m \in \mathbb{N}^+$
- The number of columns to use for ciphertext matrices $n \in \mathbb{N}^+$ s.t. $2 \leq n \leq \nu$

Require: $N = mn$

Operation:

- 1: $x \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}(p, q, \mathbf{pk}, \mathbf{ck}, \vec{C}, \vec{C}', \mathbf{c}_A))$ ▷ See algorithm 3.8 and algorithm 5.5
 - 2: $y \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}(\mathbf{c}_B, p, q, \mathbf{pk}, \mathbf{ck}, \vec{C}, \vec{C}', \mathbf{c}_A))$
 - 3: $z \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}("\text{1''}, \mathbf{c}_B, p, q, \mathbf{pk}, \mathbf{ck}, \vec{C}, \vec{C}', \mathbf{c}_A))$ ▷ Both $\vec{C}$ and $\vec{C}'$ are passed as vectors
 - 4: $\mathbf{Zneg} \leftarrow \text{Transpose}(\text{ToMatrix}(\{-z\}_{i=1}^N, m, n))$ ▷ Vector of length N , with all values being $q - z$. ▷ See algorithm 9.14 and algorithm 9.13
 - 5: $\mathbf{c}_{-z} \leftarrow \text{GetCommitmentMatrix}(\mathbf{Zneg}, \vec{0}, \mathbf{ck})$ ▷ See algorithm 9.8
 - 6: $\mathbf{c}_D \leftarrow \mathbf{c}_A^y \mathbf{c}_B$
 - 7: $b \leftarrow \prod_{i=0}^{N-1} yi + x^i - z$
 - 8: $\mathbf{pStatement} \leftarrow (\mathbf{c}_D \mathbf{c}_{-z}, b)$
 - 9: $\text{productVerif} \leftarrow \text{VerifyProductArgument}(\mathbf{pStatement}, \text{productArgument})$ ▷ See algorithm 9.19
 - 10: $\vec{x} \leftarrow \{x^i\}_{i=0}^{N-1}$
 - 11: $\mathbf{C} \leftarrow \text{GetCiphertextVectorExponentiation}(\vec{C}, \vec{x})$ ▷ See algorithm 8.7
 - 12: $\mathbf{mStatement} \leftarrow (\text{ToMatrix}(\vec{C}', m, n), \mathbf{C}, \mathbf{c}_B)$ ▷ See algorithm 9.13
 - 13: $\text{multiVerif} \leftarrow \text{VerifyMultiExponentiationArgument}(\mathbf{mStatement}, \text{multiExponentiationArgument})$ ▷ See algorithm 9.16
 - 14: **if** $\text{productVerif} \wedge \text{multiVerif}$ **then**
 - 15: **return** $\top$
 - 16: **else**
 - 17: **return** $\perp$
 - 18: **end if**
-

Output:

The result of the verification: $\top$ if the verification is successful, $\perp$ otherwise.

Test values for the algorithm 9.12 are provided in the attached [verify-shuffle-argument.json](#) file.

One of the key features of Bayer-Groth's [2] minimal shuffle argument is the transformation of a vector of ciphertexts into a $m \times n$ matrix, by means of which a prover's computation can be optimized. Therefore, the ciphertexts, received as a vector, need to be organized into a matrix. This will be achieved by setting $M_{i,j} = \vec{v}_{ni+j}$. Similarly, the exponents in the matrices A and B need to be arranged into matrices so that the other algorithms get the expected values. However, since exponents' matrices have their dimensions transposed with respect to the ciphertexts, we obtain $M_{i,j} = \vec{v}_{i+nj}$.

For completeness, we describe below the operations of organizing the elements of a vector into a matrix and the transposition of a matrix.

Algorithm 9.13 ToMatrix: Convert a vector of elements to a $m \times n$ matrix

Input:

- a vector of elements $\vec{v} \in \mathbb{D}^N$
- the number of requested rows $m \in \mathbb{N}^+$
- the number of requested columns $n \in \mathbb{N}^+$

Require: $N = m \cdot n$

Operation:

- 1: **for** $i \in [0, m)$ **do**
 - 2: **for** $j \in [0, n)$ **do**
 - 3: $M_{i,j} \leftarrow \vec{v}_{ni+j}$
 - 4: **end for**
 - 5: **end for**
-

Output:

The matrix $M = (M_{i,j})_{i,j=0}^{m,n} \in \mathbb{D}^{m \times n}$

Algorithm 9.14 Transpose: Transpose a $m \times n$ matrix to a $n \times m$ matrix

Input:

- a matrix of elements $M \in \mathbb{D}^{m \times n}$ s.t. $m, n > 0$
-

Operation:

- 1: **for** $i \in [0, n)$ **do**
 - 2: **for** $j \in [0, m)$ **do**
 - 3: $N_{i,j} \leftarrow M_{j,i}$
 - 4: **end for**
 - 5: **end for**
-

Output:

The matrix $N = (N_{i,j})_{i,j=0}^{n,m} \in \mathbb{D}^{n \times m}$

9.3.2 Multi-Exponentiation Argument

Given ciphertexts $C_{0,0}, \dots, C_{m-1,n-1}$ and C , each $\in \mathbb{H}_\ell$, algorithm 9.15 generates an argument of knowledge of the openings to the commitments $\vec{c}_A$ to $A = \{a_{i,j}\}_{i,j=1}^{n,m}$ such that

$$C = \text{GetCiphertext}(\vec{1}, \rho, \mathbf{pk}) \cdot \prod_{i=0} \vec{C}_i^{\vec{a}_{i+1}}$$

$$\vec{c}_A = \text{GetCommitmentMatrix}(A, \vec{r}, \mathbf{ck})$$

where $\vec{C}_i = (C_{i,0}, \dots, C_{i,n-1})$ and $\vec{a}_j = (a_{1,j}, \dots, a_{n,j})^T$, that is $\vec{C}_i$ refers to the i^{th} row of the matrix, whereas $\vec{a}_j$ refers to the j^{th} column. Furthermore, we use 0-based indexing for C here, which is consistent with the rest of this document, but 1-based indexing for a above, as well as $\vec{a}$ and $\mathbf{r}$ below, allowing for the generation of $\vec{a}_0$ and r_0 within the pseudocode.

Algorithm 9.15 GetMultiExponentiationArgument: Compute a multi-exponentiation argument

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
 Group generator $g \in \mathbb{G}_q$
 A multi-recipient public key $\mathbf{pk} \in \mathbb{G}_q^k$
 A commitment key $\mathbf{ck} = (h, g_1, \dots, g_\nu) \in \mathbb{C}_\nu$

Input:

The statement composed of
 - ciphertext matrix $(\vec{C}_0, \dots, \vec{C}_{m-1}) \in \mathbb{H}_\ell^{m \times n}$ s.t. $0 < \ell \leq k$ ▷ $\vec{C}_i$ refers to the i^{th} row
 - ciphertext $C \in \mathbb{H}_\ell$
 - commitment vector $\vec{c}_A \in \mathbb{G}_q^m$
 The witness composed of
 - matrix $A = (\vec{a}_1, \dots, \vec{a}_m) \in \mathbb{Z}_q^{n \times m}$ s.t. $n \leq \nu$ ▷ $\vec{a}_j$ refers to the j^{th} column
 - exponents $\mathbf{r} = (r_1, \dots, r_m) \in \mathbb{Z}_q^m$
 - exponent $\rho \in \mathbb{Z}_q$

Require: $C = \text{GetCiphertext}(\vec{1}, \rho, \mathbf{pk}) \cdot \prod_{i=0}^{m-1} \vec{C}_i^{\vec{a}_{i+1}}$ ▷ Vector of 1s of length ℓ ▷ See algorithm 8.5, algorithm 8.7, algorithm 8.8

Require: $\vec{c}_A = \text{GetCommitmentMatrix}(A, \vec{r}, \mathbf{ck})$ ▷ See algorithm 9.8

Require: $n, m > 0$

Operation:

1: $\vec{a}_0 \leftarrow \text{GenRandomVector}(q, n)$ ▷ See algorithm 5.2
 2: $r_0 \leftarrow \text{GenRandomInteger}(q)$ ▷ See algorithm 5.1
 3: $(b_0, \dots, b_{2 \cdot m-1}) \leftarrow \text{GenRandomVector}(q, 2 \cdot m)$
 4: $(s_0, \dots, s_{2 \cdot m-1}) \leftarrow \text{GenRandomVector}(q, 2 \cdot m)$
 5: $(\tau_0, \dots, \tau_{2 \cdot m-1}) \leftarrow \text{GenRandomVector}(q, 2 \cdot m)$
 6: $b_m \leftarrow 0$
 7: $s_m \leftarrow 0$
 8: $\tau_m \leftarrow \rho$ ▷ Ensuring $c_{B_m} = \text{GetCommitment}(0, 0, \mathbf{ck})$ and
 $\text{GetCiphertext}(\vec{g}^{b_m}, \tau_m, \mathbf{pk}) = \text{GetCiphertext}(\vec{1}, \rho, \mathbf{pk})$
 9: $c_{A_0} \leftarrow \text{GetCommitment}(\vec{a}_0, r_0, \mathbf{ck})$ ▷ See algorithm 9.7
 10: $(D_0, \dots, D_{2 \cdot m-1}) \leftarrow \text{GetDiagonalProducts}((\vec{C}_0, \dots, \vec{C}_{m-1}), (\vec{a}_0, \dots, \vec{a}_m))$ ▷ See algorithm 9.17
 11: **for** $k \in [0, 2 \cdot m)$ **do**
 12: $c_{B_k} \leftarrow \text{GetCommitment}((b_k), s_k, \mathbf{ck})$
 13: $E_k \leftarrow \text{GetCiphertext}(\vec{g}^{b_k}, \tau_k, \mathbf{pk}) \cdot D_k$ ▷ See algorithm 8.5, we take a vector of messages of length ℓ each having value g^{b_k}
 14: **end for**
 15: $x \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}(p, q, \mathbf{pk}, \mathbf{ck}, (\vec{C}_i)_{i=0}^{m-1}, C, \vec{c}_A, c_{A_0}, (c_{B_k})_{k=0}^{2 \cdot m-1}, (E_k)_{k=0}^{2 \cdot m-1}))$ ▷ See algorithm 3.8 and algorithm 5.5
 ▷ All operations below are performed modulo q
 16: $\vec{a} \leftarrow \vec{a}_0 + \sum_{i=1}^m x^i \vec{a}_i$
 17: $r \leftarrow r_0 + \sum_{i=1}^m x^i r_i$
 18: $b \leftarrow b_0 + \sum_{k=1}^{2 \cdot m-1} x^k b_k$
 19: $s \leftarrow s_0 + \sum_{k=1}^{2 \cdot m-1} x^k s_k$
 20: $\tau \leftarrow \tau_0 + \sum_{k=1}^{2 \cdot m-1} x^k \tau_k$

Output:

$\text{multiExponentiationArgument} = (c_{A_0}, (c_{B_k})_{k=0}^{2 \cdot m-1}, (E_k)_{k=0}^{2 \cdot m-1}, \vec{a}, r, b, s, \tau) \in \mathbb{G}_q \times \mathbb{G}_q^{2m} \times \mathbb{H}_\ell^{2m} \times \mathbb{Z}_q^n \times \mathbb{Z}_q \times \mathbb{Z}_q \times \mathbb{Z}_q \times \mathbb{Z}_q$

In the following pseudocode algorithm, we verify that a provided Multi-Exponentiation argument adequately supports the corresponding statement.

Algorithm 9.16 VerifyMultiExponentiationArgument: Verify a multi-exponentiation argument

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
 Group generator $g \in \mathbb{G}_q$
 A multi-recipient public key $\mathbf{pk} \in \mathbb{G}_q^k$
 A commitment key $\mathbf{ck} = (h, g_1, \dots, g_\nu) \in \mathbb{C}_\nu$

Input:

The **statement** composed of
 - ciphertext matrix $(\vec{C}_0, \dots, \vec{C}_{m-1}) \in \mathbb{H}_\ell^{m \times n}$ $\triangleright \vec{C}_i$ refers to the i^{th} row
 - ciphertext $C \in \mathbb{H}_\ell$
 - commitment vector $\vec{c}_A = (c_{A_1}, \dots, c_{A_m}) \in \mathbb{G}_q^m$
 The **argument** composed of
 - the commitment $c_{A_0} \in \mathbb{G}_q$
 - the commitment vector $\mathbf{c}_B = (c_{B_0}, \dots, c_{B_{2 \cdot m-1}}) \in \mathbb{G}_q^{2 \cdot m}$
 - the ciphertext vector $\mathbf{E} = (E_0, \dots, E_{2 \cdot m-1}) \in \mathbb{H}_\ell^{2 \cdot m}$
 - the vector of exponents $\vec{a} = (a_0, \dots, a_{n-1}) \in \mathbb{Z}_q^n$
 - the exponent $r \in \mathbb{Z}_q$
 - the exponent $b \in \mathbb{Z}_q$
 - the exponent $s \in \mathbb{Z}_q$
 - the exponent $\tau \in \mathbb{Z}_q$

Require: $n, m > 0$

Operation:

```

1:  $x \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}(p, q, \mathbf{pk}, \mathbf{ck}, \{\vec{C}_i\}_{i=0}^{m-1}, C, \vec{c}_A, c_{A_0}, \{c_{B_k}\}_{k=0}^{2 \cdot m-1}, \{E_k\}_{k=0}^{2 \cdot m-1}))$   $\triangleright$  See algorithm 3.8 and algorithm 5.5
2:  $\text{verifCbm} \leftarrow c_{B_m} = 1$ 
3:  $\text{verifEm} \leftarrow E_m = C$ 

4:  $\text{prodCa} \leftarrow c_{A_0} \prod_{i=1}^m c_{A_i}^{x^i}$ 
5:  $\text{commA} \leftarrow \text{GetCommitment}(\vec{a}, r, \mathbf{ck})$   $\triangleright$  See algorithm 9.7
6:  $\text{verifA} \leftarrow \text{prodCa} = \text{commA}$ 

7:  $\text{prodCb} \leftarrow c_{B_0} \prod_{k=1}^{2 \cdot m-1} c_{B_k}^{x^k}$ 
8:  $\text{commB} \leftarrow \text{GetCommitment}((b), s, \mathbf{ck})$ 
9:  $\text{verifB} \leftarrow \text{prodCb} = \text{commB}$ 

10:  $\text{prodE} \leftarrow E_0 \prod_{k=1}^{2 \cdot m-1} E_k^{x^k}$ 
11:  $\text{encryptedGb} \leftarrow \text{GetCiphertext}(\vec{g}^b, \tau, \mathbf{pk})$   $\triangleright$  See algorithm 8.5, we take a vector of messages of length  $\ell$  each having value  $g^b$ 
12:  $\text{prodC} \leftarrow \prod_{i=0}^{m-1} \text{GetCiphertextVectorExponentiation}(\vec{C}_i, x^{(m-i)-1} \vec{a})$   $\triangleright$  See algorithm 8.7
13:  $\text{verifEC} \leftarrow \text{prodE} = \text{GetCiphertextProduct}(\text{encryptedGb}, \text{prodC})$   $\triangleright$  See algorithm 8.8

14: if  $\text{verifCbm} \wedge \text{verifEm} \wedge \text{verifA} \wedge \text{verifB} \wedge \text{verifEC}$  then
15:   return  $\top$ 
16: else
17:   return  $\perp$ 
18: end if

```

Output:

The result of the verification: $\top$ if the verification is successful, $\perp$ otherwise.

Test values for the algorithm 9.16 are provided in the attached [verify-multiexp-argument.json](#) file.

Algorithm 9.17 GetDiagonalProducts: Compute the products of the diagonals of a ciphertext matrix

Context:

- Group modulus $p \in \mathbb{P}$
- Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
- Group generator $g \in \mathbb{G}_q$
- A multi-recipient public key $\mathbf{pk} \in \mathbb{G}_q^k$

Input:

- Ciphertext matrix $C = (\vec{C}_0, \dots, \vec{C}_{m-1}) \in \mathbb{H}_\ell^{m \times n}$ $\triangleright \vec{C}_i$ refers to the i^{th} row
 - Exponent matrix $A = (\vec{a}_0, \dots, \vec{a}_m) \in \mathbb{Z}_q^{n \times (m+1)}$ $\triangleright \vec{a}_j$ refers to the j^{th} column
-

Operation:

- 1: **for** $k \in [0, 2 \cdot m)$ **do**
 - 2: $d_k \leftarrow \underbrace{(1, \dots, 1)}_{\ell+1 \text{ times}}$ $\triangleright$ Neutral element of ciphertext multiplication
 - 3: **if** $k < m$ **then**
 - 4: lowerbound $\leftarrow m - k - 1$
 - 5: upperbound $\leftarrow m$
 - 6: **else**
 - 7: lowerbound $\leftarrow 0$
 - 8: upperbound $\leftarrow 2 \cdot m - k$
 - 9: **end if**
 - 10: **for** $i \in [\text{lowerbound}, \text{upperbound})$ **do**
 - 11: $j \leftarrow k - m + i + 1$
 - 12: $d_k \leftarrow \text{GetCiphertextProduct}(d_k, \text{GetCiphertextVectorExponentiation}(\vec{C}_i, \vec{a}_j))$ $\triangleright$
 - See algorithm 8.8 and algorithm 8.7
 - 13: **end for**
 - 14: **end for**
-

Output:

- Diagonal products $D = (d_0, \dots, d_{2 \cdot m-1}) \in \mathbb{H}_\ell^{2 \cdot m}$
-

9.3.3 Product Argument

The following algorithm provides an argument that a set of committed values have a particular product.

More precisely, given commitments $\vec{c}_A = (c_{A_0}, \dots, c_{A_m})$ to $A = \{a_{i,j}\}_{i,j=0}^{n-1,m-1}$ and a value b , we want to give an argument of knowledge for $\prod_{i=0}^{n-1} \prod_{j=0}^{m-1} a_{i,j} = b$.

We will first compute a commitment c_b as follows:

$$c_b = \text{GetCommitment} \left(\left(\prod_{j=0}^{m-1} a_{0,j}, \dots, \prod_{j=0}^{m-1} a_{n-1,j} \right), s, \mathbf{ck} \right)$$

We will then give an argument that c_b is correct, using a Hadamard argument (see section 9.3.4), showing that the values committed in c_b are indeed the result of the Hadamard product of the values committed in c_A . Additionally, we will show that the value b is the product of the values committed in c_b , using a Single Value Product Argument (see section 9.3.6).

If the number of ciphertexts to be shuffled is prime and they cannot be arranged into a matrix, $m = 1$ and $n = N$, the Hadamard Product is trivially equal to the first (and only) column of the matrix and we can omit the Hadamard argument, calling the Single Value Product argument directly.

Algorithm 9.18 GetProductArgument: Computes a Product Argument

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
 Group generator $g \in \mathbb{G}_q$
 A multi-recipient public key $\mathbf{pk} \in \mathbb{G}_q^k$
 A commitment key $\mathbf{ck} = (h, g_1, \dots, g_\nu) \in \mathbb{C}_\nu$

Input:

The statement composed of
 - commitments $\vec{c}_A = (c_{A_1}, \dots, c_{A_m}) \in \mathbb{G}_q^m$
 - the product $b \in \mathbb{Z}_q$
 The witness composed of
 - the matrix $A \in \mathbb{Z}_q^{n \times m}$
 - the exponents $\vec{r} \in \mathbb{Z}_q^m$

Require: $2 \leq n \leq \nu$

Require: $m > 0$

Require: $\vec{c}_A = \text{GetCommitmentMatrix}(A, \vec{r}, \mathbf{ck})$

▷ See algorithm 9.8

Require: $b = \prod_{i=0}^{n-1} \prod_{j=0}^{m-1} a_{i,j} \bmod q$

Operation:

```

1: if  $m > 1$  then
2:    $s \leftarrow \text{GenRandomInteger}(q)$                                 ▷ See algorithm 5.1
3:   for  $i \in [0, n)$  do
4:      $b_i \leftarrow \prod_{j=0}^{m-1} a_{i,j}$ 
5:   end for
6:    $c_b \leftarrow \text{GetCommitment}((b_0, \dots, b_{n-1}), s, \mathbf{ck})$       ▷ See algorithm 9.7
7:    $\text{hStatement} \leftarrow (\vec{c}_A, c_b)$ 
8:    $\text{hWitness} \leftarrow (A, (b_0, \dots, b_{n-1}), \vec{r}, s)$ 
9:    $\text{hadamardArg} \leftarrow \text{GetHadamardArgument}(\text{hStatement}, \text{hWitness})$   ▷ See algorithm 9.20
10:   $\text{sStatement} \leftarrow (c_b, b)$ 
11:   $\text{sWitness} \leftarrow ((b_0, \dots, b_{n-1}), s)$ 
12:   $\text{singleValueProdArg} \leftarrow \text{GetSingleValueProductArgument}(\text{sStatement}, \text{sWitness})$   ▷ See algorithm 9.25
13: else
14:   $\text{sStatement} \leftarrow (c_{A_1}, b)$ 
15:   $\text{sWitness} \leftarrow (\vec{a}_0, r_0)$ 
16:   $\text{singleValueProdArg} \leftarrow \text{GetSingleValueProductArgument}(\text{sStatement}, \text{sWitness})$   ▷ See algorithm 9.25
17: end if

```

Output:

```

18: if  $m > 1$  then
19:    $\text{productArg} = (c_b, \text{hadamardArg}, \text{singleValueProdArg}) \in \mathbb{G}_q \times (\mathbb{G}_q^{3m+3} \times \mathbb{Z}_q^{2n+3}) \times (\mathbb{G}_q^3 \times \mathbb{Z}_q^{2n+2})$ 
19: else
20:    $\text{productArg} = \text{singleValueProdArg} \in \mathbb{G}_q^3 \times \mathbb{Z}_q^{2n+2}$ 
20: end if

```

In the following pseudocode algorithm, we verify if a provided Product argument supports the corresponding statement.

Algorithm 9.19 VerifyProductArgument: Verify a Product argument

Context:

- Group modulus $p \in \mathbb{P}$
- Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
- Group generator $g \in \mathbb{G}_q$
- A multi-recipient public key $\mathbf{pk} \in \mathbb{G}_q^k$
- A commitment key $\mathbf{ck} = (h, g_1, \dots, g_\nu) \in \mathbb{C}_\nu$

Input:

- The statement composed of
- commitments $\vec{c}_A = (c_{A_1}, \dots, c_{A_m}) \in \mathbb{G}_q^m$
- the product $b \in \mathbb{Z}_q$

The argument composed of

- the commitment $c_b \in \mathbb{G}_q$ ▷ omitted if $m = 1$
- a hadamardArg $\in \mathbb{G}_q^{3m+3} \times \mathbb{Z}_q^{2n+3}$ ▷ omitted if $m = 1$
- a singleValueProductArg $\in \mathbb{G}_q^3 \times \mathbb{Z}_q^{2n+2}$

Require: $2 \leq n \leq \nu$

Require: $m > 0$

Operation:

```

1: if  $m > 1$  then
2:    $\mathbf{hStatement} \leftarrow (\vec{c}_A, c_b)$ 
3:    $\mathbf{sStatement} \leftarrow (c_b, b)$ 
4:   if VerifyHadamardArgument( $\mathbf{hStatement}$ , hadamardArg)  $\wedge$ 
5:   VerifySingleValueProductArgument( $\mathbf{sStatement}$ , singleValueProductArg) then
      ▷ See algorithm 9.21 and algorithm 9.26
      return  $\top$ 
6:   else
      return  $\perp$ 
7:   end if
8: else
9:    $\mathbf{sStatement} \leftarrow (c_{A_1}, b)$ 
10:  if VerifySingleValueProductArgument( $\mathbf{sStatement}$ , singleValueProductArg) then
      ▷ See algorithm 9.26
      return  $\top$ 
11:  else
      return  $\perp$ 
12:  end if
13: end if

```

Output:

The result of the verification: $\top$ if the verification is successful, $\perp$ otherwise.

Test values for the algorithm 9.19 are provided in the attached [verify-p-argument.json](#) file.

9.3.4 Hadamard Argument

The operations given in algorithm 9.20 are more readable using vector notation. That is, we note $\vec{a}$ for the vector $(a_0, \dots, a_{n-1}) \in \mathbb{Z}_q^n$. By extension, we denote matrix $a_{0,0}, \dots, a_{n-1,m-1}$ as $\vec{a}_0, \dots, \vec{a}_{m-1}$ where each vector corresponds to a column of the matrix.

In the following algorithm, we generate an argument of knowledge of the openings $\vec{a}_0, \dots, \vec{a}_{m-1}$ and $\vec{b}$ to the commitments $\mathbf{c}_A$ and c_b , such that:

$$\begin{aligned} \mathbf{c}_A &= \text{GetCommitmentMatrix}((\vec{a}_0, \dots, \vec{a}_{m-1}), \mathbf{r}, \mathbf{ck}) \\ c_b &= \text{GetCommitment}((b_0, \dots, b_{n-1}), s, \mathbf{ck}) \\ b_i &= \prod_{j=0}^{m-1} a_{i,j} \text{ for } i = 0, \dots, n-1 \end{aligned}$$

where the product in the last line matches the entry-wise product, also known as Hadamard product.

The subsequent pseudocode algorithm verifies if a provided Hadamard argument supports the corresponding statement.

Algorithm 9.20 GetHadamardArgument: Computes a Hadamard Argument

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
 Group generator $g \in \mathbb{G}_q$
 A multi-recipient public key $\mathbf{pk} \in \mathbb{G}_q^k$
 A commitment key $\mathbf{ck} = (h, g_1, \dots, g_\nu) \in \mathbb{C}_\nu$

Input:

The statement composed of
 - commitment $\mathbf{c}_A = (c_{A_0}, \dots, c_{A_{m-1}}) \in \mathbb{G}_q^m$
 - commitment $c_b \in \mathbb{G}_q$
 The witness composed of
 - matrix $A = (\vec{a}_0, \dots, \vec{a}_{m-1}) \in \mathbb{Z}_q^{n \times m}$
 - vector $\mathbf{b} \in \mathbb{Z}_q^n$
 - exponents $\mathbf{r} = (r_0, \dots, r_{m-1}) \in \mathbb{Z}_q^m$
 - exponent $s \in \mathbb{Z}_q$

Require: $m \geq 2$

▷ Hadamard product only makes sense for $m \geq 2$

Require: $0 < n \leq \nu$

Require: $\mathbf{c}_A = \text{GetCommitmentMatrix}(A, \mathbf{r}, \mathbf{ck})$

▷ See algorithm 9.8

Require: $c_b = \text{GetCommitment}(\mathbf{b}, s, \mathbf{ck})$

▷ See algorithm 9.7

Require: $\vec{b} = \prod_{j=0}^{m-1} \vec{a}_j$

▷ Uses the Hadamard product, ie $b_i = \prod_{j=0}^{m-1} a_{i,j}$

Operation:

```

1: for  $j \in [0, m)$  do
2:    $\vec{b}_j \leftarrow \prod_{i=0}^j \vec{a}_i$ 
3: end for
4:  $s_0 \leftarrow r_0$ 
5: if  $m > 2$  then
6:    $(s_1, \dots, s_{m-2}) \leftarrow \text{GenRandomVector}(q, m-2)$ 
7: end if
8:  $s_{m-1} \leftarrow s$ 
9:  $c_{B_0} \leftarrow c_{A_0}$ 
10: for  $j \in [1, m-1)$  do
11:    $c_{B_j} \leftarrow \text{GetCommitment}(\vec{b}_j, s_j, \mathbf{ck})$ 
12: end for
13:  $c_{B_{m-1}} \leftarrow c_b$ 
14:  $x \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}(p, q, \mathbf{pk}, \mathbf{ck}, \mathbf{c}_A, c_b, (c_{B_0}, \dots, c_{B_{m-1}})))$ 
15:  $y \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}("1", p, q, \mathbf{pk}, \mathbf{ck}, \mathbf{c}_A, c_b, (c_{B_0}, \dots, c_{B_{m-1}})))$ 
    * :  $\mathbb{Z}_q^n \times \mathbb{Z}_q^n \rightarrow \mathbb{Z}_q$ , see algorithm 9.10
16: for  $i \in [0, m-1)$  do
17:    $\vec{d}_i = x^{i+1} \vec{b}_i$ 
18:    $c_{D_i} = c_{B_i}^{x^{i+1}}$ 
19:    $t_i = x^{i+1} s_i$ 
20: end for
21:  $\vec{d} \leftarrow \sum_{i=1}^{m-1} x^i \vec{b}_i$ 
22:  $c_D \leftarrow \prod_{i=1}^{m-1} c_{B_i}^{x^i}$ 
23:  $t \leftarrow \sum_{i=1}^{m-1} x^i s_i$ 
24:  $-\vec{1} \leftarrow (q-1, \dots, q-1) \in \mathbb{Z}_q^n$ 
25:  $c_{-1} \leftarrow \text{GetCommitment}(-\vec{1}, 0, \mathbf{ck})$ 
26: statement  $\leftarrow ((c_{A_1}, \dots, c_{A_{m-1}}, c_{-1}), (c_{D_0}, \dots, c_{D_{m-2}}, c_D), y)$ 
27: witness  $\leftarrow ((\vec{a}_1, \dots, \vec{a}_{m-1}, -\vec{1}), (\vec{d}_0, \dots, \vec{d}_{m-2}, \vec{d}), (r_1, \dots, r_{m-1}, 0), (t_0, \dots, t_{m-2}, t))$ 
28: zeroArg  $\leftarrow \text{GetZeroArgument}(\text{statement}, \text{witness})$ 

```

▷ Which implies that $\vec{b}_0 = \vec{a}_0$ and $\vec{b}_{m-1} = \vec{b}$

▷ Thus ensuring that $\text{GetCommitment}(\vec{b}_0, s_0, \mathbf{ck}) = c_{A_0}$

▷ See algorithm 5.2

▷ Thus ensuring that $\text{GetCommitment}(\vec{b}_{m-1}, s_{m-1}, \mathbf{ck}) = c_b$

▷ See algorithm 9.7

▷ See algorithm 3.8 and algorithm 5.5

▷ Use y to define

▷ All exponentiations of x below are performed modulo q

▷ See algorithm 9.7

▷ See algorithm 9.22

▷ Provide an argument that $\sum_{i=0}^{m-2} \vec{a}_{i+1} \star \vec{d}_i - \vec{1} \star \vec{d} = 0$

Output:

hadamardArgument $= ((c_{B_0}, \dots, c_{B_{m-1}}), \text{zeroArg}) \in \mathbb{G}_q^m \times (\mathbb{G}_q \times \mathbb{G}_q \times \mathbb{G}_q^{2m+1} \times \mathbb{Z}_q^n \times \mathbb{Z}_q^n \times \mathbb{Z}_q \times \mathbb{Z}_q \times \mathbb{Z}_q)$

Algorithm 9.21 VerifyHadamardArgument: Verifies a Hadamard Argument

Context:

- Group modulus $p \in \mathbb{P}$
- Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
- Group generator $g \in \mathbb{G}_q$
- A multi-recipient public key $\mathbf{pk} \in \mathbb{G}_q^k$
- A commitment key $\mathbf{ck} = (h, g_1, \dots, g_\nu) \in \mathbb{C}_\nu$

Input:

- The **statement** composed of
 - commitment $\mathbf{c}_A = (c_{A_0}, \dots, c_{A_{m-1}}) \in \mathbb{G}_q^m$
 - commitment $c_b \in \mathbb{G}_q$
- The **argument** composed of
 - commitment vector $\mathbf{c}_B = (c_{B_0}, \dots, c_{B_{m-1}}) \in \mathbb{G}_q^m$
 - a zero argument, composed of
 - commitment $c_{A_0} \in \mathbb{G}_q$
 - commitment $c_{B_m} \in \mathbb{G}_q$
 - commitment vector $\mathbf{c}_d \in \mathbb{G}_q^{2 \cdot m + 1}$
 - exponent vector $\mathbf{a}' \in \mathbb{Z}_q^n$
 - exponent vector $\mathbf{b}' \in \mathbb{Z}_q^n$
 - exponent $r' \in \mathbb{Z}_q$
 - exponent $s' \in \mathbb{Z}_q$
 - exponent $t' \in \mathbb{Z}_q$

Require: $n > 0$

Operation:

- 1: $x \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}(p, q, \mathbf{pk}, \mathbf{ck}, \mathbf{c}_A, c_b, (c_{B_0}, \dots, c_{B_{m-1}})))$ ▷ See algorithm 3.8 and algorithm 5.5
 - 2: $y \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}("\text{1''}", p, q, \mathbf{pk}, \mathbf{ck}, \mathbf{c}_A, c_b, (c_{B_0}, \dots, c_{B_{m-1}})))$
 - 3: **for** $i \in [0, m-1]$ **do**
 - 4: $c_{D_i} \leftarrow c_{B_i}^{x^{i+1}}$
 - 5: **end for**
 - 6: $c_D \leftarrow \prod_{i=1}^{m-1} c_{B_i}^{x^i}$
 - 7: $-\vec{1} \leftarrow (q-1, \dots, q-1) \in \mathbb{Z}_q^n$
 - 8: $c_{-1} \leftarrow \text{GetCommitment}(-\vec{1}, 0, \mathbf{ck})$ ▷ See algorithm 9.7
 - 9: $\text{zeroStatement} \leftarrow ((c_{A_1}, \dots, c_{A_{m-1}}, c_{-1}), (c_{D_0}, \dots, c_{D_{m-2}}, c_D), y)$
 - 10: $\text{zeroArgument} \leftarrow (c_{A_0}, c_{B_m}, \mathbf{c}_d, \mathbf{a}', \mathbf{b}', r', s', t')$
 - 11: **if** $c_{B_0} = c_{A_0} \wedge c_{B_{m-1}} = c_b \wedge \text{VerifyZeroArgument}(\text{zeroStatement}, \text{zeroArgument})$ **then** ▷ See algorithm 9.23
 - return** $\top$
 - 12: **else**
 - return** $\perp$
 - 13: **end if**
-

Output:

The result of the verification: $\top$ if the verification is successful, $\perp$ otherwise.

Test values for the algorithm 9.21 are provided in the attached [verify-h-argument.json](#) file.

9.3.5 Zero Argument

In the following algorithm, we generate an argument of knowledge of the values $\mathbf{a}_1, \mathbf{b}_0, \dots, \mathbf{a}_m, \mathbf{b}_{m-1}$ such that $\sum_{i=1}^m \mathbf{a}_i \star \mathbf{b}_{i-1} = 0$.

Algorithm 9.22 GetZeroArgument: Computes a Zero Argument

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
 Group generator $g \in \mathbb{G}_q$
 A multi-recipient public key $\mathbf{pk} \in \mathbb{G}_q^k$
 A commitment key $\mathbf{ck} = (h, g_1, \dots, g_\nu) \in \mathbb{C}_\nu$

Input:

The statement composed of
 - commitments $\mathbf{c}_A \in \mathbb{G}_q^m$
 - commitments $\mathbf{c}_B \in \mathbb{G}_q^m$
 - the value $y \in \mathbb{Z}_q$ defining the bilinear mapping $\star$ ▷ See algorithm 9.10
 The witness composed of
 - matrix $A = (\bar{a}_1, \dots, \bar{a}_m) \in \mathbb{Z}_q^{n \times m}$ ▷ The $\bar{a}_i$ values correspond to the columns of A
 - matrix $B = (\bar{b}_0, \dots, \bar{b}_{m-1}) \in \mathbb{Z}_q^{n \times m}$ ▷ The $\bar{b}_i$ values correspond to the columns of B
 - vector of exponents $\mathbf{r} = (r_1, \dots, r_m) \in \mathbb{Z}_q^m$
 - vector of exponents $\mathbf{s} = (s_0, \dots, s_{m-1}) \in \mathbb{Z}_q^m$

Require: $\mathbf{c}_A = \text{GetCommitmentMatrix}(A, \mathbf{r}, \mathbf{ck})$ ▷ See algorithm 9.8

Require: $\mathbf{c}_B = \text{GetCommitmentMatrix}(B, \mathbf{s}, \mathbf{ck})$

Require: $\sum_{i=1}^m \mathbf{a}_i \star \mathbf{b}_{i-1} = 0$

Require: $n, m > 0$

Operation:

```

1:  $\bar{a}_0 \leftarrow \text{GenRandomVector}(q, n)$  ▷ See algorithm 5.2
2:  $\bar{b}_m \leftarrow \text{GenRandomVector}(q, n)$ 
3:  $r_0 \leftarrow \text{GenRandomInteger}(q)$  ▷ See algorithm 5.1
4:  $s_m \leftarrow \text{GenRandomInteger}(q)$ 
5:  $c_{A_0} \leftarrow \text{GetCommitment}(\bar{a}_0, r_0, \mathbf{ck})$  ▷ See algorithm 9.7
6:  $c_{B_m} \leftarrow \text{GetCommitment}(\bar{b}_m, s_m, \mathbf{ck})$ 
7:  $\mathbf{d} = (d_0, \dots, d_{2 \cdot m}) \leftarrow \text{ComputeDVector}((\bar{a}_0, \dots, \bar{a}_m), (\bar{b}_0, \dots, \bar{b}_m), y)$  ▷ See algorithm 9.24
8:  $\mathbf{t} \leftarrow \text{GenRandomVector}(q, 2 \cdot m + 1)$ 
9:  $t_{m+1} \leftarrow 0$ 
10:  $\mathbf{c}_d \leftarrow \text{GetCommitmentVector}((d_0, \dots, d_{2 \cdot m}), \mathbf{t}, \mathbf{ck})$  ▷ See algorithm 9.9
11:  $x \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}(p, q, \mathbf{pk}, \mathbf{ck}, c_{A_0}, c_{B_m}, \mathbf{c}_d, \mathbf{c}_B, \mathbf{c}_A))$  ▷ See algorithm 3.8 and
    algorithm 5.5 ▷ Below this point, all operations are performed modulo  $q$ 

12: for  $j \in [0, n)$  do
13:    $\mathbf{a}'_j \leftarrow \sum_{i=0}^m x^i \cdot \bar{a}_{j,i}$ 
14:    $\mathbf{b}'_j \leftarrow \sum_{i=0}^m x^{m-i} \cdot \bar{b}_{j,i}$ 
15: end for
16:  $r' \leftarrow \sum_{i=0}^m x^i \cdot r_i$ 
17:  $s' \leftarrow \sum_{i=0}^m x^{m-i} \cdot s_i$ 
18:  $t' \leftarrow \sum_{i=0}^{2 \cdot m} x^i \cdot t_i$ 
    
```

Output:

$\text{zeroArgument} = (c_{A_0}, c_{B_m}, \mathbf{c}_d, \mathbf{a}', \mathbf{b}', r', s', t') \in \mathbb{G}_q \times \mathbb{G}_q \times \mathbb{G}_q^{2m+1} \times \mathbb{Z}_q^n \times \mathbb{Z}_q^n \times \mathbb{Z}_q \times \mathbb{Z}_q \times \mathbb{Z}_q$

In the following algorithm, we verify if a provided zero argument supports the corresponding statement. We conform to the convention of using the symbol $\top$ for true and $\perp$ for false.

Algorithm 9.23 VerifyZeroArgument: Verifies a Zero Argument

Context:

- Group modulus $p \in \mathbb{P}$
- Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
- Group generator $g \in \mathbb{G}_q$
- A multi-recipient public key $\mathbf{pk} \in \mathbb{G}_q^k$
- A commitment key $\mathbf{ck} = (h, g_1, \dots, g_\nu) \in \mathbb{C}_\nu$

Input:

- The **statement** composed of
 - commitments $\mathbf{c}_A = (c_{A_1}, \dots, c_{A_m}) \in \mathbb{G}_q^m$
 - commitments $\mathbf{c}_B = (c_{B_0}, \dots, c_{B_{m-1}}) \in \mathbb{G}_q^m$
 - the value $y \in \mathbb{Z}_q$ defining the bilinear mapping $\star$ ▷ See algorithm 9.10
 - The **argument** composed of
 - the commitment $c_{A_0} \in \mathbb{G}_q$
 - the commitment $c_{B_m} \in \mathbb{G}_q$
 - the commitment vector $\mathbf{c}_d = (c_{d_0}, \dots, c_{d_{2 \cdot m}}) \in \mathbb{G}_q^{2 \cdot m + 1}$
 - the exponent vector $\mathbf{a}' \in \mathbb{Z}_q^n$
 - the exponent vector $\mathbf{b}' \in \mathbb{Z}_q^n$
 - the exponent $r' \in \mathbb{Z}_q$
 - the exponent $s' \in \mathbb{Z}_q$
 - the exponent $t' \in \mathbb{Z}_q$
-

Operation:

- 1: $x \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}(p, q, \mathbf{pk}, \mathbf{ck}, c_{A_0}, c_{B_m}, \mathbf{c}_d, \mathbf{c}_B, \mathbf{c}_A))$ ▷ See algorithm 3.8 and algorithm 5.5
 - 2: $\text{verifCd} \leftarrow c_{d_{m+1}} = 1$
 - 3: $\text{prodCa} \leftarrow \prod_{i=0}^m c_{A_i}^{x^i}$ ▷ The exponentiations of x are computed modulo q , whereas the product and the exponentiations of commitments are computed modulo p
 - 4: $\text{commA} \leftarrow \text{GetCommitment}(\mathbf{a}', r', \mathbf{ck})$ ▷ See algorithm 9.7
 - 5: $\text{verifA} \leftarrow \text{prodCa} = \text{commA}$
 - 6: $\text{prodCb} \leftarrow \prod_{i=0}^m c_{B_{m-i}}^{x^i}$ ▷ The exponentiations of x are computed modulo q , whereas the product and the exponentiations of commitments are computed modulo p
 - 7: $\text{commB} \leftarrow \text{GetCommitment}(\mathbf{b}', s', \mathbf{ck})$
 - 8: $\text{verifB} \leftarrow \text{prodCb} = \text{commB}$
 - 9: $\text{prodCd} \leftarrow \prod_{i=0}^{2 \cdot m} c_{d_i}^{x^i}$ ▷ The exponentiations of x are computed modulo q , whereas the product and the exponentiations of commitments are computed modulo p
 - 10: $\text{prod} \leftarrow \mathbf{a}' \star \mathbf{b}'$ ▷ Using algorithm 9.10 with value y
 - 11: $\text{commD} \leftarrow \text{GetCommitment}(\text{prod}, t', \mathbf{ck})$
 - 12: $\text{verifD} \leftarrow \text{prodCd} = \text{commD}$
 - 13: **if** $\text{verifCd} \wedge \text{verifA} \wedge \text{verifB} \wedge \text{verifD}$ **then**
 return $\top$
 - 14: **else**
 return $\perp$
 - 15: **end if**
-

Output:

The result of the verification: $\top$ if the verification is successful, $\perp$ otherwise.

Test values for the algorithm 9.23 are provided in the attached [verify-za-argument.json](#) file.

Algorithm 9.24 ComputeDVector: Compute the vector $\mathbf{d}$ for the algorithm 9.22

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
 Group generator $g \in \mathbb{G}_q$

Input:

First matrix $A = (\vec{a}_0, \dots, \vec{a}_m) \in \mathbb{Z}_q^{n \times (m+1)}$
 Second matrix $B = (\vec{b}_0, \dots, \vec{b}_m) \in \mathbb{Z}_q^{n \times (m+1)}$
 Value $y \in \mathbb{Z}_q$

Require: $n, m > 0$

Operation:

```

1: for  $k \in [0, 2 \cdot m]$  do
2:    $d_k \leftarrow 0$ 
3:   for  $i \in [\max(0, k - m), m]$  do
4:      $j \leftarrow (m - k) + i$ 
5:     if  $j > m$  then
6:       break from loop and proceed with next  $k$ 
7:     end if
8:      $d_k \leftarrow d_k + \vec{a}_i \star \vec{b}_j$  ▷ See algorithm 9.10, addition is modulo  $q$ 
9:   end for
10: end for

```

Output:

$\mathbf{d} = (d_0, \dots, d_{2 \cdot m}) \in \mathbb{Z}_q^{2 \cdot m + 1}$

9.3.6 Single Value Product Argument

In the following algorithm we generate an argument of knowledge of the opening $(\mathbf{a}, r)$ where $\mathbf{a} = (a_0, \dots, a_{n-1})$ s.t. $c_a = \text{GenCommitment}(\mathbf{a}, r)$ and $b = \prod_{i=0}^{n-1} a_i \mod q$.

Algorithm 9.25 GetSingleValueProductArgument: Computes a Single Value Product Argument

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
 Group generator $g \in \mathbb{G}_q$
 A multi-recipient public key $\mathbf{pk} \in \mathbb{G}_q^k$
 A commitment key $\mathbf{ck} = (h, g_1, \dots, g_\nu) \in \mathbb{C}_\nu$

Input:

The statement composed of
 - commitment $c_a \in \mathbb{G}_q$
 - the product $b \in \mathbb{Z}_q$
 The witness composed of
 - vector $\mathbf{a} = (a_0, \dots, a_{n-1}) \in \mathbb{Z}_q^n$
 - the randomness $r \in \mathbb{Z}_q$

Require: $n \geq 2$

Require: $c_a = \text{GenCommitment}(\mathbf{a}, r, \mathbf{ck})$

▷ See algorithm 9.7

Require: $b = \prod_{i=0}^{n-1} a_i \mod q$

Operation:

```

1: for  $k \in [0, n)$  do
2:    $b_k \leftarrow \prod_{i=0}^k a_i \mod q$ 
3: end for
4:  $(d_0, \dots, d_{n-1}) \leftarrow \text{GenRandomVector}(q, n)$ 
5:  $r_d \leftarrow \text{GenRandomInteger}(q)$ 
6:  $\delta_0 \leftarrow d_0$ 
7: if  $n > 2$  then
8:    $(\delta_1, \dots, \delta_{n-2}) \leftarrow \text{GenRandomVector}(q, n-2)$ 
9: end if
10:  $\delta_{n-1} \leftarrow 0$ 
11:  $s_0 \leftarrow \text{GenRandomInteger}(q)$ 
12:  $s_x \leftarrow \text{GenRandomInteger}(q)$ 
13: for  $k \in [0, n-1)$  do
14:    $\delta'_k \leftarrow -\delta_k d_{k+1} \mod q$ 
15:    $\Delta_k \leftarrow \delta_{k+1} - a_{k+1} \delta_k - b_k d_{k+1} \mod q$ 
16: end for
17:  $c_d \leftarrow \text{GetCommitment}((d_0, \dots, d_{n-1}), r_d, \mathbf{ck})$ 
18:  $c_\delta \leftarrow \text{GetCommitment}((\delta'_0, \dots, \delta'_{n-2}), s_0, \mathbf{ck})$ 
19:  $c_\Delta \leftarrow \text{GetCommitment}((\Delta_0, \dots, \Delta_{n-2}), s_x, \mathbf{ck})$ 
20:  $x \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}(p, q, \mathbf{pk}, \mathbf{ck}, c_\Delta, c_\delta, c_d, b, c_a))$ 
21: for  $k \in [0, n)$  do
22:    $\tilde{a}_k \leftarrow x \cdot a_k + d_k \mod q$ 
23:    $\tilde{b}_k \leftarrow x \cdot b_k + \delta_k \mod q$ 
24: end for
25:  $\tilde{r} \leftarrow x \cdot r + r_d \mod q$ 
26:  $\tilde{s} \leftarrow x \cdot s_x + s_0 \mod q$ 

```

▷ See algorithm 9.7

▷ See algorithm 3.8 and algorithm 5.5

Output:

$\text{singleValueProdArg} = (c_d, c_\delta, c_\Delta, (\tilde{a}_0, \dots, \tilde{a}_{n-1}), (\tilde{b}_0, \dots, \tilde{b}_{n-1}), \tilde{r}, \tilde{s}) \in \mathbb{G}_q \times \mathbb{G}_q \times \mathbb{G}_q \times \mathbb{Z}_q^n \times \mathbb{Z}_q^n \times \mathbb{Z}_q \times \mathbb{Z}_q$

In the following pseudocode algorithm, we verify if a provided Single Value Product argument supports the corresponding statement.

Algorithm 9.26 VerifySingleValueProductArgument: Verifies a Single Value Product Argument

Context:

- Group modulus $p \in \mathbb{P}$
- Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
- Group generator $g \in \mathbb{G}_q$
- A multi-recipient public key $\mathbf{pk} \in \mathbb{G}_q^k$
- A commitment key $\mathbf{ck} = (h, g_1, \dots, g_\nu) \in \mathbb{C}_\nu$

Input:

- The statement composed of
 - commitment $c_a \in \mathbb{G}_q$
 - the product $b \in \mathbb{Z}_q$
 - The argument composed of
 - the commitment $c_d \in \mathbb{G}_q$
 - the commitment $c_\delta \in \mathbb{G}_q$
 - the commitment $c_\Delta \in \mathbb{G}_q$
 - the exponent vector $\tilde{a} = (\tilde{a}_0, \dots, \tilde{a}_{n-1}) \in \mathbb{Z}_q^n, n \geq 2$
 - the exponent vector $\tilde{b} = (\tilde{b}_0, \dots, \tilde{b}_{n-1}) \in \mathbb{Z}_q^n$
 - the exponent $\tilde{r} \in \mathbb{Z}_q$
 - the exponent $\tilde{s} \in \mathbb{Z}_q$
-

Operation:

- 1: $x \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}(p, q, \mathbf{pk}, \mathbf{ck}, c_\Delta, c_\delta, c_d, b, c_a))$ ▷ See algorithm 3.8 and algorithm 5.5
 - 2: $\text{prodCa} \leftarrow c_a^x \cdot c_d$
 - 3: $\text{commA} \leftarrow \text{GetCommitment}(\tilde{a}, \tilde{r}, \mathbf{ck})$ ▷ See algorithm 9.7
 - 4: $\text{verifA} \leftarrow \text{prodCa} = \text{commA}$
 - 5: $\text{prodDelta} \leftarrow c_\Delta^x \cdot c_\delta$
 - 6: **for** $i \in [0, n-1)$ **do**
 - 7: $e_i \leftarrow x \cdot \tilde{b}_{i+1} - \tilde{b}_i \cdot \tilde{a}_{i+1}$
 - 8: **end for**
 - 9: $\text{commDelta} \leftarrow \text{GetCommitment}((e_0, \dots, e_{n-2}), \tilde{s}, \mathbf{ck})$
 - 10: $\text{verifDelta} \leftarrow \text{prodDelta} = \text{commDelta}$
 - 11: $\text{verifB} \leftarrow \tilde{b}_0 = \tilde{a}_0 \wedge \tilde{b}_{n-1} = x \cdot b$
 - 12: **if** $\text{verifA} \wedge \text{verifDelta} \wedge \text{verifB}$ **then**
 - 13: **return** $\top$
 - 13: **else**
 - 14: **return** $\perp$
 - 14: **end if**
-

Output:

The result of the verification: $\top$ if the verification is successful, $\perp$ otherwise.

Test values for the algorithm 9.26 are provided in the attached [verify-svp-argument.json](#) file.

10 Zero-Knowledge Proofs

10.1 Introduction

This section introduces various Zero-Knowledge Proofs of Knowledge, based on the work by Maurer [23]. We extensively document and formalize the zero-knowledge proof system’s security—including the non-interactive case—in the computational proof [29]. In each case, the idea is to make a **statement**, consisting of a homomorphism $\phi : \mathbb{G}_1 \rightarrow \mathbb{G}_2$ and an image y and provide a Zero-Knowledge Proof of the Pre-image w such that $y = \phi(w)$. We name that pre-image the **witness**.

While such proofs are usually interactive, we rely on the Fiat-Shamir transform to turn them non-interactive. We use the hash function described in algorithm 5.5. The proof consists of the following steps:

- draw $b \in \mathbb{G}_1$ at random
- compute $c = \phi(b)$
- compute $e = \text{RecursiveHash}(\phi, y, c, \text{auxiliaryData})$
- compute $z = b \star w^e$ (where $\star$ is the group operation for $\mathbb{G}_1$, and exponentiation is the repetition of that operation)
- output $\pi = (e, z)$

The verification can be summarized as:

- compute $x = \phi(z)$
- compute $c' = x \otimes y^{-e}$ (where $\otimes$ is the group operation for $\mathbb{G}_2$, and exponentiation is the repetition of that operation)
- if and only if $\text{RecursiveHash}(\phi, y, c', \text{auxiliaryData}) = e$, the proof is valid

Each type of proof is a specialization of the generic prove and verify algorithm.

10.2 Schnorr Proof

In this section we provide a proof of knowledge of a discrete logarithm, also known as a Schnorr proof. Given the values x, y, g and p such that $x \equiv \log_g(y) \pmod{p}$, we want to prove knowledge of x without revealing its value.

In this case, the phi-function is $x \mapsto g^x \bmod p$, with domain $(\mathbb{Z}_q, +)$ and co-domain $(\mathbb{G}_q, \times)$. As such, the operations given as $\star$ consist of additions modulo q and the “exponentiation” used in the computation of z is a multiplication; whereas the operation noted as $\otimes$ is a multiplication modulo p , and the exponentiation given in the computation of c' is a modular exponentiation in $\mathbb{G}_q$.

Algorithm 10.1 ComputePhiSchnorr: Compute the phi-function for a Schnorr proof

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
Base $g \in \mathbb{G}_q \setminus \{1\}$

Input:

An exponent $x \in \mathbb{Z}_q$

Operation:

1: $y \leftarrow g^x \bmod p$
2: **return** y

Output:

The power $y \in \mathbb{G}_q$

Algorithm 10.2 GenSchnorrProof: Generate a proof of knowledge of a discrete logarithm

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
Base $g \in \mathbb{G}_q \setminus \{1\}$

Input:

The witness – a secret exponent $x \in \mathbb{Z}_q$
The statement – a power $y \in \mathbb{G}_q$ s.t. $y = g^x$
An array of optional additional information $\mathbf{i}_{\text{aux}} \in (\mathbb{A}_{UCS^*})^s, s \in \mathbb{N}$

Operation:

- 1: $b \leftarrow \text{GenRandomInteger}(q)$ ▷ See algorithm 5.1
- 2: $c \leftarrow \text{ComputePhiSchnorr}(b)$ ▷ See algorithm 10.1
- 3: $\mathbf{f} \leftarrow (p, q, g)$
- 4: $\mathbf{h}_{\text{aux}} \leftarrow (\text{"SchnorrProof"}, \mathbf{i}_{\text{aux}})$ ▷ If $\mathbf{i}_{\text{aux}}$ is empty, we omit it
- 5: $e \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}(\mathbf{f}, y, c, \mathbf{h}_{\text{aux}}))$ ▷ See algorithms 3.8 and 5.5
- 6: $z \leftarrow b + e \cdot x \bmod q$

Output:

Proof $(e, z) \in \mathbb{Z}_q \times \mathbb{Z}_q$

Algorithm 10.3 VerifySchnorr: Verifies the validity of a Schnorr proof

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
 Base $g \in \mathbb{G}_q \setminus \{1\}$

Input:

The proof $(e, z) \in \mathbb{Z}_q \times \mathbb{Z}_q$
 The statement – a power $y \in \mathbb{G}_q$
 An array of optional additional information $\mathbf{i}_{\text{aux}} \in (\mathbb{A}_{UCS^*})^s, s \in \mathbb{N}$

Operation:

```

1:  $x \leftarrow \text{ComputePhiSchnorr}(z)$  ▷ See algorithm 10.1
2:  $\mathbf{f} \leftarrow (p, q, g)$ 
3:  $c' \leftarrow x \cdot y^{-e} \bmod p$ 
4:  $\mathbf{h}_{\text{aux}} \leftarrow (\text{"SchnorrProof"}, \mathbf{i}_{\text{aux}})$  ▷ If  $\mathbf{i}_{\text{aux}}$  is empty, we omit it
5:  $e' \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}(\mathbf{f}, y, c', \mathbf{h}_{\text{aux}}))$  ▷ See algorithms 3.8 and 5.5
6: if  $e = e'$  then
    return  $\top$ 
7: else
    return  $\perp$ 
8: end if
    
```

Output:

The result of the verification: $\top$ if the verification is successful, $\perp$ otherwise.

Test values for the algorithm 10.3 are provided in the attached [verify-schnorr.json](#) file.

10.3 Decryption Proof

We prove that a decryption matches the message encrypted under the advertised public key. In this case, the phi-function maps our witness—the private key—to the public key and the decryption of the ciphertext. Hence, we define the phi-function as shown in algorithm 10.4.

Algorithm 10.4 ComputePhiDecryption: Compute the phi-function for decryption

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
 Group generator $g \in \mathbb{G}_q$

Input:

Preimage $(x_0, \dots, x_{\ell-1}) \in \mathbb{Z}_q^\ell$
 Base $\gamma \in \mathbb{G}_q$

Operation:

```

1: for  $i \in [0, \ell)$  do
2:    $y_i \leftarrow g^{x_i}$ 
3:    $y_{\ell+i} \leftarrow \gamma^{x_i}$ 
4: end for

```

$\triangleright y_i = \mathbf{pk}_i$ when $x_i = \mathbf{sk}_i$
 $\triangleright y_{\ell+i} = g^{\mathbf{sk}_i \cdot r} = \frac{\phi_i}{m_i}$ when $\gamma = g^r$ and $x_i = \mathbf{sk}_i$

$\triangleright$ All symbols used in the comments above are aligned with algorithms 8.4 and 8.5

Output:

The image $(y_0, \dots, y_{2\cdot\ell-1}) \in \mathbb{G}_q^{2\cdot\ell}$

This algorithm implies that for the multi-recipient ElGamal key pair $(\mathbf{pk}, \mathbf{sk})$ and the valid decryption $m = (m_0, \dots, m_{\ell-1})$ of the ciphertext $(\gamma, \phi_0, \dots, \phi_{\ell-1})$, the computation of the $\text{ComputePhiDecryption}(\mathbf{sk}, \gamma)$ would yield $(\mathbf{pk}_0, \dots, \mathbf{pk}_{\ell-1}, \frac{\phi_0}{m_0}, \dots, \frac{\phi_{\ell-1}}{m_{\ell-1}})$.

Generating and verifying decryption proofs The algorithms below are the adaptations of the general case presented in section 10.1, with explicit domains and operations. Our ϕ -function defined in algorithm 10.4 has domain $(\mathbb{Z}_q^\ell, +)$ and co-domain $(\mathbb{G}_q^{2\ell}, \times)$. Therefore the operations given as $\star$ will be replaced with addition (modulo q), and the “exponentiation” used in the computation of z is actually a multiplication; whereas the operation denoted by $\otimes$ is multiplication (modulo p) and the exponentiation used in the computation of c' is a modular exponentiation in $\mathbb{G}_q$.

Algorithm 10.5 GenDecryptionProof: Generate a proof of validity for the provided decryption

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
Group generator $g \in \mathbb{G}_q$

Input:

A multi-recipient ciphertext $\mathbf{C} = (\gamma, \phi_0, \dots, \phi_{\ell-1}) \in \mathbb{H}_\ell$
A multi-recipient key pair $(\mathbf{pk}, \mathbf{sk}) \in \mathbb{G}_q^k \times \mathbb{Z}_q^k$
A multi-recipient message $\mathbf{m} = (m_0, \dots, m_{\ell-1}) \in \mathbb{G}_q^\ell$ s.t. $\mathbf{m} = \text{GetMessage}(\mathbf{C}, \mathbf{sk})$
An array of optional additional information $\mathbf{i}_{\text{aux}} \in (\mathbb{A}_{UCS^*})^s, s \in \mathbb{N}$

Require: $0 < \ell \leq k$

Operation:

```

1:  $\mathbf{b} \leftarrow \text{GenRandomVector}(q, \ell)$  ▷ See algorithm 5.2
2:  $\mathbf{c} \leftarrow \text{ComputePhiDecryption}(\mathbf{b}, \gamma)$  ▷ See algorithm 10.4
3:  $\mathbf{f} \leftarrow (p, q, g, \gamma)$ 
4: for  $i \in [0, \ell)$  do
5:    $y_i \leftarrow \mathbf{pk}_i$ 
6:    $y_{\ell+i} \leftarrow \frac{\phi_i}{m_i}$ 
7: end for
8:  $\mathbf{h}_{\text{aux}} \leftarrow (\text{"DecryptionProof"}, (\phi_0, \dots, \phi_{\ell-1}), \mathbf{m}, \mathbf{i}_{\text{aux}})$  ▷ If  $\mathbf{i}_{\text{aux}}$  is empty, we omit it
9:  $e \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}(\mathbf{f}, (y_0, \dots, y_{2 \cdot \ell - 1}), \mathbf{c}, \mathbf{h}_{\text{aux}}))$  ▷ See
   algorithms 3.8 and 5.5
10:  $\mathbf{sk}' \leftarrow (\mathbf{sk}_0, \dots, \mathbf{sk}_{\ell-1})$ 
11:  $\mathbf{z} \leftarrow \mathbf{b} + e \cdot \mathbf{sk}'$ 

```

Output:

Proof $(e, \mathbf{z}) \in \mathbb{Z}_q \times \mathbb{Z}_q^\ell$

Algorithm 10.6 VerifyDecryption: Verifies the validity of a decryption proof

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
 Group generator $g \in \mathbb{G}_q$

Input:

A multi-recipient ciphertext $C = (\gamma, \phi_0, \dots, \phi_{\ell-1}) \in \mathbb{H}_\ell$
 A multi-recipient public key $\mathbf{pk} = (\mathbf{pk}_0, \dots, \mathbf{pk}_{k-1}) \in \mathbb{G}_q^k$
 A multi-recipient message $\mathbf{m} = (m_0, \dots, m_{\ell-1}) \in \mathbb{G}_q^\ell$ ▷ We expect
 $\mathbf{m} = \text{GetMessage}(\mathbf{C}, \mathbf{sk})$
 The proof $(e, \mathbf{z}) \in \mathbb{Z}_q \times \mathbb{Z}_q^\ell$
 An array of optional additional information $\mathbf{i}_{\text{aux}} \in (\mathbb{A}_{UCS^*})^s, s \in \mathbb{N}$

Require: $0 < \ell \leq k$

Operation:

```

1:  $\mathbf{x} \leftarrow \text{ComputePhiDecryption}(\mathbf{z}, \gamma)$  ▷ See algorithm 10.4
2:  $\mathbf{f} \leftarrow (p, q, g, \gamma)$ 
3: for  $i \in [0, \ell)$  do
4:    $y_i \leftarrow \mathbf{pk}_i$ 
5:    $y_{\ell+i} \leftarrow \frac{\phi_i}{m_i}$ 
6: end for
7: for  $i \in [0, 2 \cdot \ell)$  do
8:    $c'_i \leftarrow x_i y_i^{-e}$ 
9: end for
10:  $\mathbf{h}_{\text{aux}} \leftarrow (\text{"DecryptionProof"}, (\phi_0, \dots, \phi_{\ell-1}), \mathbf{m}, \mathbf{i}_{\text{aux}})$  ▷ If  $\mathbf{i}_{\text{aux}}$  is empty, we omit it
11:  $e' \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}(\mathbf{f}, (y_0, \dots, y_{2 \cdot \ell-1}), (c'_0, \dots, c'_{2 \cdot \ell-1}), \mathbf{h}_{\text{aux}}))$  ▷ See
    algorithms 3.8 and 5.5
12: if  $e = e'$  then
13:   return  $\top$ 
14: else
15:   return  $\perp$ 
16: end if

```

Output:

The result of the verification: $\top$ if the verification is successful, $\perp$ otherwise.

Test values for the algorithm 10.6 are provided in the attached [verify-decryption.json](#) file.

10.4 Exponentiation Proof

We prove that the same secret exponent is used for a vector of exponentiations. In this case, the phi-function maps our witness—the secret exponent—to the exponentiation of a given vector of bases. We define the phi-function as shown in algorithm 10.7.

Algorithm 10.7 ComputePhiExponentiation: Compute the phi-function for exponentiation

Context:

Group modulus $p \in \mathbb{P}$

Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$

Input:

Preimage $x \in \mathbb{Z}_q$

Bases $(g_0, \dots, g_{n-1}) \in \mathbb{G}_q^n$ s.t. $n \in \mathbb{N}^+$

Operation:

- 1: **for** $i \in [0, n)$ **do**
 - 2: $y_i \leftarrow g_i^x \bmod p$
 - 3: **end for**
 - 4: **return** $(y_0, \dots, y_{n-1})$
-

Output:

$\mathbf{y} = (y_0, \dots, y_{n-1}) \in \mathbb{G}_q^n$

Generating and verifying exponentiation proofs The algorithms below are the adaptations of the general case presented in section 10.1, with explicit domains and operations. Our phi-function defined in algorithm 10.7 has domain $(\mathbb{Z}_q, +)$ and co-domain $(\mathbb{G}_q^n, \times)$. Therefore the operations given as $\star$ will be replaced with addition (modulo q), and the “exponentiation” used in the computation of z is a multiplication; whereas the operation denoted by $\otimes$ is multiplication (modulo p) and the exponentiation used in the computation of c' is a modular exponentiation in $\mathbb{G}_q$.

Algorithm 10.8 GenExponentiationProof: Generate a proof of validity for the provided exponentiation

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$

Input:

A vector of bases $\mathbf{g} = (g_0, \dots, g_{n-1}) \in \mathbb{G}_q^n$ s.t. $n \in \mathbb{N}^+$
The witness – a secret exponent $x \in \mathbb{Z}_q$
The statement – a vector of exponentiations $\mathbf{y} = (y_0, \dots, y_{n-1}) \in \mathbb{G}_q^n$ s.t. $y_i = g_i^x$
An array of optional additional information $\mathbf{i}_{\text{aux}} \in (\mathbb{A}_{UCS}^*)^s, s \in \mathbb{N}$

Operation:

- 1: $b \leftarrow \text{GenRandomInteger}(q)$ ▷ See algorithm 5.1
 - 2: $\mathbf{c} \leftarrow \text{ComputePhiExponentiation}(b, \mathbf{g})$ ▷ See algorithm 10.7
 - 3: $\mathbf{f} \leftarrow (p, q, \mathbf{g})$
 - 4: $\mathbf{h}_{\text{aux}} \leftarrow (\text{"ExponentiationProof"}, \mathbf{i}_{\text{aux}})$ ▷ If $\mathbf{i}_{\text{aux}}$ is empty, we omit it
 - 5: $e \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}(\mathbf{f}, \mathbf{y}, \mathbf{c}, \mathbf{h}_{\text{aux}}))$ ▷ See algorithms 3.8 and 5.5
 - 6: $z \leftarrow b + e \cdot x \mod q$
-

Output:

Proof $(e, z) \in \mathbb{Z}_q \times \mathbb{Z}_q$

Algorithm 10.9 VerifyExponentiation: Verifies the validity of an exponentiation proof

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$

Input:

A vector of bases $\mathbf{g} = (g_0, \dots, g_{n-1}) \in \mathbb{G}_q^n$ s.t. $n \in \mathbb{N}^+$
The **statement** – a vector of exponentiations $\mathbf{y} = (y_0, \dots, y_{n-1}) \in \mathbb{G}_q^n$
The **proof** $(e, z) \in \mathbb{Z}_q \times \mathbb{Z}_q$
An array of optional additional information $\mathbf{i}_{\text{aux}} \in (\mathbb{A}_{UCS}^*)^s, s \in \mathbb{N}$

Operation:

```

1:  $\mathbf{x} \leftarrow \text{ComputePhiExponentiation}(z, \mathbf{g})$  ▷ See algorithm 10.7
2:  $\mathbf{f} \leftarrow (p, q, \mathbf{g})$ 
3: for  $i \in [0, n)$  do
4:    $c'_i \leftarrow x_i \cdot y_i^{-e}$ 
5: end for
6:  $\mathbf{h}_{\text{aux}} \leftarrow (\text{"ExponentiationProof"}, \mathbf{i}_{\text{aux}})$  ▷ If  $\mathbf{i}_{\text{aux}}$  is empty, we omit it
7:  $e' \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}(\mathbf{f}, \mathbf{y}, (c'_0, \dots, c'_{n-1}), \mathbf{h}_{\text{aux}}))$  ▷ See algorithms 3.8 and 5.5
8: if  $e = e'$  then
   return  $\top$ 
9: else
   return  $\perp$ 
10: end if

```

Output:

The result of the verification: $\top$ if the verification is successful, $\perp$ otherwise.

Test values for the algorithm 10.9 are provided in the attached [verify-exponentiation.json](#) file.

10.5 Plaintext Equality Proof

We prove that two encryptions under different keys correspond to the same plaintext. The ciphertexts are written as $\mathbf{c} = (c_0, c_1) = (g^r, h^r m)$ and $\mathbf{c}' = (c'_0, c'_1) = (g^{r'}, h'^{r'} m)$, where g is the generator, h and h' are the public keys, and m is the same message in both cases. In this case, the phi-function is defined by the primes p and q , defining $\mathbb{G}_q$, as well as the generator g and the public keys h and h' , as follows:

$$\begin{aligned} \phi_{\text{PlaintextEquality}} : \mathbb{Z}_q^2 &\rightarrow \mathbb{G}_q^3 \\ \phi_{\text{PlaintextEquality}}(x, x') &= (g^x, g^{x'}, \frac{h^x}{h'^{x'}}) \end{aligned}$$

This implies that $\phi_{\text{PlaintextEquality}}(r, r') = (c_0, c'_0, \frac{c_1}{c'_1})$, if and only if the message is the same in both encryptions.

Algorithm 10.10 ComputePhiPlaintextEquality: Compute the phi-function for plaintext equality

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
Group generator $g \in \mathbb{G}_q$

Input:

Preimage $(x, x') \in \mathbb{Z}_q^2$
First public key $h \in \mathbb{G}_q$
Second public key $h' \in \mathbb{G}_q$

Operation:

1: **return** $(g^x, g^{x'}, \frac{h^x}{h'^{x'}})$ ▷ All exponentiations performed modulo p

Output:

The image $(g^x, g^{x'}, \frac{h^x}{h'^{x'}}) \in \mathbb{G}_q^3$

Generating and verifying plaintext equality proofs The algorithms below are the adaptations of the general case presented in section 10.1, with explicit domains and operations. Our phi-function defined in algorithm 10.10 has domain $(\mathbb{Z}_q^2, +)$ and co-domain $(\mathbb{G}_q^3, \times)$. Therefore the operations given as $\star$ will be replaced with addition (modulo q), and the “exponentiation” used in the computation of z is a multiplication; whereas the operation denoted by $\otimes$ is multiplication (modulo p) and the exponentiation used in the computation of c' is a modular exponentiation in $\mathbb{G}_q$.

Algorithm 10.11 GenPlaintextEqualityProof: Generate a proof of equality of the plaintext corresponding to the two provided encryptions

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
 Group generator $g \in \mathbb{G}_q$

Input:

The first ciphertext $\mathbf{C} = (c_0, c_1) \in \mathbb{G}_q^2$
 The second ciphertext $\mathbf{C}' = (c'_0, c'_1) \in \mathbb{G}_q^2$
 The first public key $h \in \mathbb{G}_q$
 The second public key $h' \in \mathbb{G}_q$
 The witness—the randomness used in the encryptions— $(r, r') \in \mathbb{Z}_q^2$
 An array of optional additional information $\mathbf{i}_{\text{aux}} \in (\mathbb{A}_{UCS^*})^s, s \in \mathbb{N}$

Operation:

- 1: $(b_1, b_2) \leftarrow \text{GenRandomVector}(q, 2)$ ▷ See algorithm 5.2
 - 2: $\mathbf{c} \leftarrow \text{ComputePhiPlaintextEquality}((b_1, b_2), h, h')$ ▷ See algorithm 10.10
 - 3: $\mathbf{f} \leftarrow (p, q, g, h, h')$
 - 4: $\mathbf{y} \leftarrow (c_0, c'_0, \frac{c_1}{c'_1})$
 - 5: $\mathbf{h}_{\text{aux}} \leftarrow (\text{"PlaintextEqualityProof"}, c_1, c'_1, \mathbf{i}_{\text{aux}})$ ▷ If $\mathbf{i}_{\text{aux}}$ is empty, we omit it
 - 6: $e \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}(\mathbf{f}, \mathbf{y}, \mathbf{c}, \mathbf{h}_{\text{aux}}))$ ▷ See algorithms 3.8 and 5.5
 - 7: $\mathbf{z} \leftarrow (b_1 + e \cdot r, b_2 + e \cdot r')$
-

Output:

Proof $(e, \mathbf{z}) \in \mathbb{Z}_q \times \mathbb{Z}_q^2$

Algorithm 10.12 `VerifyPlaintextEquality`: Verifies the validity of a plaintext equality proof

Context:

- Group modulus $p \in \mathbb{P}$
- Group cardinality $q \in \mathbb{P}$ s.t. $p = 2 \cdot q + 1$
- Group generator $g \in \mathbb{G}_q$

Input:

- The first ciphertext $\mathbf{C} = (c_0, c_1) \in \mathbb{G}_q^2$
 - The second ciphertext $\mathbf{C}' = (c'_0, c'_1) \in \mathbb{G}_q^2$
 - The first public key $h \in \mathbb{G}_q$
 - The second public key $h' \in \mathbb{G}_q$
 - The proof $(e, \mathbf{z}) \in \mathbb{Z}_q \times \mathbb{Z}_q^2$
 - An array of optional additional information $\mathbf{i}_{\text{aux}} \in (\mathbb{A}_{UCS^*})^s, s \in \mathbb{N}$
-

Operation:

- 1: $\mathbf{x} \leftarrow \text{ComputePhiPlaintextEquality}(\mathbf{z}, h, h')$ ▷ See algorithm 10.10
 - 2: $\mathbf{f} \leftarrow (p, q, g, h, h')$
 - 3: $\mathbf{y} \leftarrow (c_0, c'_0, \frac{c_1}{c'_1})$
 - 4: $\mathbf{c}' \leftarrow \mathbf{x} \cdot \mathbf{y}^{-e}$
 - 5: $\mathbf{h}_{\text{aux}} \leftarrow (\text{"PlaintextEqualityProof"}, c_1, c'_1, \mathbf{i}_{\text{aux}})$ ▷ If $\mathbf{i}_{\text{aux}}$ is empty, we omit it
 - 6: $e' \leftarrow \text{ByteArrayToInteger}(\text{RecursiveHash}(\mathbf{f}, \mathbf{y}, \mathbf{c}', \mathbf{h}_{\text{aux}}))$ ▷ See algorithms 3.8 and 5.5
 - 7: **if** $e = e'$ **then**
return $\top$
 - 8: **else**
return $\perp$
 - 9: **end if**
-

Output:

The result of the verification: $\top$ if the verification is successful, $\perp$ otherwise.

Test values for the algorithm 10.12 are provided in the attached [verify-plaintext-equality.json](#) file.

Acknowledgements

Swiss Post is thankful to all security researchers for their contributions and the opportunity to improve the system's security guarantees. In particular, we want to thank the following experts for their reviews or suggestions reported on our [Gitlab repository](#). We list them here in alphabetical order:

- Aleksander Essex (Western University Canada)
- Rolf Haenni, Reto Koenig, Philipp Locher, Eric Dubuis (Bern University of Applied Sciences)
- Thomas Edmund Haines (Australian National University)
- Sarah Jamie Lewis (Open privacy)
- Pascal Junod (modulo p SA)
- Sylvain Pelissier (Kudelski Security Research)
- Olivier Pereira (Universtité catholique Louvain)
- Ruben Santamarta
- Vanessa Teague (Thinking Cybersecurity)
- François Weissbaum, Patrick Liniger (Swiss Armed Forces Command Support Organisation)

References

- [1] Elaine Barker. NIST SP 800-57 Part 1, Revision 5, Recommendation for Key Management. National Institute of Standards & Technology (NIST), 2020.
- [2] Stephanie Bayer and Jens Groth. Efficient Zero-Knowledge Argument for Correctness of a Shuffle. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, pages 263–280, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
- [3] Mihir Bellare, Alexandra Boldyreva, and Jessica Staddon. Randomness Re-use in Multi-recipient Encryption Schemes. In *Public Key Cryptography - PKC 2003, 6th International Workshop on Theory and Practice in Public Key Cryptography, Miami, FL, USA, January 6-8, 2003, Proceedings*, pages 85–99, 2003.
- [4] Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich. Argon2: New Generation of Memory-Hard Functions for Password Hashing and Other Applications. In *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 292–302. IEEE, 2016.
- [5] Alex Biryukov, Daniel Dinu, Dmitry Khovratovich, and Simon Josefsson. Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications. RFC 9106, September 2021.
- [6] Lily Chen, Dustin Moody, Andrew Regenscheid, and Angela Robinson. FIPS 186-5. Digital Signature Standard (DSS). National Institute of Standards & Technology (NIST), 2023.
- [7] David Cooper, Stefan Santesson, Stephen Farrell, Sharon Boeyen, Russell Housley, W Timothy Polk, et al. Internet X. 509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile. *RFC*, 5280:1–151, 2008.
- [8] Die Schweizerische Bundeskanzlei (BK). Federal Chancellery Ordinance on Electronic Voting (OEV), 01 July 2022.
- [9] Morris J. Dworkin. Sp 800-38d. Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC. National Institute of Standards & Technology (NIST), 2007.
- [10] Morris J. Dworkin. FIPS 202. SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions. National Institute of Standards & Technology (NIST), 2015.
- [11] Eidgenössisches Departement für auswärtige Angelegenheiten EDA. Swiss Political System - Direct Democracy. <https://www.eda.admin.ch/aboutswitzerland/en/home/politik/uebersicht/direkte-demokratie.html/>. Retrieved on 2020-07-15.

- [12] gfs.bern. Vorsichtige Offenheit im Bereich digitale Partizipation - Schlussbericht, 3 2020.
- [13] Oded Goldreich. *Foundations of Cryptography: Volume 1, Basic Tools*. Cambridge University Press, 2007.
- [14] Rolf Haenni, Eric Dubuis, Reto E Koenig, and Philipp Locher. CHVote: Sixteen Best Practices and Lessons Learned. In *International Joint Conference on Electronic Voting*, pages 95–111. Springer, 2020.
- [15] Rolf Haenni, Reto E. Koenig, Philipp Locher, and Eric Dubuis. CHVote Protocol Specification, Version 3.5. Cryptology ePrint Archive, Report 2017/325, 2023. <https://eprint.iacr.org/2017/325>.
- [16] Thomas Haines. Finding Report: SwissPost Voting System - Signature Verification. <https://gitlab.anu.edu.au/u1113289/thomas-public-paper/-/raw/a5f1c738d7e034c360dc5fccddf49ea9555a42b1/SwissPostSigningMarch2021.pdf>, November 2021. Accessed: 2022-02-23.
- [17] Thomas Haines, Rageev Goré, and Bhavesh Sharma. Did you mix me? Formally Verifying Verifiable Mix Nets in Electronic Voting.
- [18] Tetsu Iwata, Keisuke Ohashi, and Kazuhiko Minematsu. Breaking and repairing GCM security proofs. In *Annual Cryptology Conference*, pages 31–49. Springer, 2012.
- [19] Simon Josefsson et al. The Base16, Base32, and Base64 Data Encodings. Technical report, RFC 4648, October, 2006.
- [20] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. CRC press, 2020.
- [21] Donald E. Knuth. *The Art of Computer Programming, Volume 2 (3rd Ed.): Seminumerical Algorithms*. Addison-Wesley Longman Publishing Co., Inc., USA, 1997.
- [22] Hugo Krawczyk and Pasi Eronen. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). Technical report, RFC 5869, May, 2010.
- [23] Ueli Maurer. Unifying Zero-Knowledge Proofs of Knowledge. In Bart Preneel, editor, *Progress in Cryptology – AFRICACRYPT 2009*, pages 272–286, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg.
- [24] David McGrew. An Interface and Algorithms for Authenticated Encryption. RFC 5116, January 2008.
- [25] Alfred J Menezes, Paul C Van Oorschot, and Scott A Vanstone. *Handbook of Applied Cryptography*. CRC press, 2018.

- [26] Gary L Miller. Riemann’s Hypothesis and Tests for Primality. *Journal of computer and system sciences*, 13(3):300–317, 1976.
- [27] Kathleen Moriarty, Burt Kaliski, Jakob Jonsson, and Andreas Rusch. PKCS #1: RSA Cryptography Specifications Version 2.2. RFC 8017, November 2016.
- [28] Torben P. Pedersen. Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing. In *CRYPTO*, pages 129–140, 1991.
- [29] Swiss Post. Protocol of the Swiss Post Voting System. Computational Proof of Complete Verifiability and Privacy. Version 1.3.0. <https://gitlab.com/swisspost-evoting/e-voting/e-voting-documentation/-/tree/master/Protocol>, 2024.
- [30] Michael O Rabin. Probabilistic Algorithm for Testing Primality. *Journal of number theory*, 12(1):128–138, 1980.
- [31] Recommendation, ITUT. ITU-T Recommendation X. 690 ASN. 1 encoding rules: Specification of Basic Encoding Rules (BER), Canonical Encoding Rules (CER) and Distinguished Encoding Rules (DER), 2008.
- [32] Ben Smyth. A foundation for secret, verifiable elections. *IACR Cryptology ePrint Archive*, 2018:225, 2018.
- [33] Björn Terelius and Douglas Wikström. Proofs of Restricted Shuffles. In *International Conference on Cryptology in Africa*, pages 100–113. Springer, 2010.
- [34] Yiannis Tsiounis and Moti Yung. On the Security of ElGamal Based Encryption. In *Public Key Cryptography*, pages 117–134, 1998.
- [35] Francois Yergeau. RFC3629: UTF-8, a transformation format of ISO 10646, 2003.

List of Algorithms

3.1	CutToBitLength	11
3.2	Base16Encode	12
3.3	Base16Decode	12
3.4	Base32Encode	12
3.5	Base32Decode	13
3.6	Base64Encode	13
3.7	Base64Decode	13
3.8	ByteArrayToInteger	14
3.9	IntegerToByteArray	15
3.10	IntegerToFixedLengthByteArray	15
3.11	ByteLength	16
3.12	StringToByteArray	17
3.13	ByteArrayToString	17
3.14	StringToInteger	18
3.15	IntegerToString	18
3.16	Truncate	19
5.1	GenRandomInteger	22
5.2	GenRandomVector	23
5.3	GenRandomString	23
5.4	GenUniqueDecimalStrings	24
5.5	RecursiveHash	26
5.6	RecursiveHashToZq	27
5.7	RecursiveHashOfLength	28
5.8	HashAndSquare	29
5.9	KDF	30
5.10	KDFToZq	31
5.11	GenArgon2id	33
5.12	GetArgon2id	33
6.1	GenCiphertextSymmetric	35
6.2	GetPlaintextSymmetric	36
7.1	GenKeysAndCert	39
7.2	GenSignature	41
7.3	VerifySignature	42
8.1	GetEncryptionParameters	45
8.2	GetSmallPrimeGroupMembers	46
8.3	IsSmallPrime	47
8.4	GenKeyPair	48
8.5	GetCiphertext	49
8.6	GetCiphertextExponentiation	50
8.7	GetCiphertextVectorExponentiation	50
8.8	GetCiphertextProduct	51
8.9	GetMessage	52

8.10	GetPartialDecryption	52
8.11	GenVerifiableDecryptions	53
8.12	VerifyDecryptions	54
8.13	CombinePublicKeys	55
9.1	GenVerifiableShuffle	57
9.2	VerifyShuffle	58
9.3	GenShuffle	59
9.4	GenPermutation	60
9.5	GetMatrixDimensions	61
9.6	GetVerifiableCommitmentKey	63
9.7	GetCommitment	64
9.8	GetCommitmentMatrix	64
9.9	GetCommitmentVector	65
9.10	StarMap	67
9.11	GetShuffleArgument	69
9.12	VerifyShuffleArgument	70
9.13	ToMatrix	71
9.14	Transpose	71
9.15	GetMultiExponentiationArgument	73
9.16	VerifyMultiExponentiationArgument	74
9.17	GetDiagonalProducts	75
9.18	GetProductArgument	77
9.19	VerifyProductArgument	78
9.20	GetHadamardArgument	80
9.21	VerifyHadamardArgument	81
9.22	GetZeroArgument	82
9.23	VerifyZeroArgument	83
9.24	ComputeDVector	84
9.25	GetSingleValueProductArgument	85
9.26	VerifySingleValueProductArgument	86
10.1	ComputePhiSchnorr	88
10.2	GenSchnorrProof	89
10.3	VerifySchnorr	90
10.4	ComputePhiDecryption	91
10.5	GenDecryptionProof	92
10.6	VerifyDecryption	93
10.7	ComputePhiExponentiation	94
10.8	GenExponentiationProof	95
10.9	VerifyExponentiation	96
10.10	ComputePhiPlaintextEquality	97
10.11	GenPlaintextEqualityProof	98
10.12	VerifyPlaintextEquality	99

List of Figures

1	Bayer-Groth argument for the correctness of a shuffle	66
---	---	----

List of Tables

2	Security levels	8
3	Primitives and their parametrization, independent of the security level chosen	9
4	Example representations of different byte arrays	10
5	Example representations of different integers	14
6	Example representations of different strings	16
7	Example conversions of strings to integers	18
8	Profiles for Argon2	32